

**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS TIMÓTEO**

João Pedro Ferreira Duarte

**GERAÇÃO TEXT-TO-SQL VIA DESCOBERTA AUTÔNOMA DE
METADADOS: UMA ARQUITETURA PARA CRIAÇÃO E AVALIAÇÃO
DE CONSULTAS SQL GERADAS POR LLMS**

Timóteo

2026

João Pedro Ferreira Duarte

**GERAÇÃO TEXT-TO-SQL VIA DESCOBERTA AUTÔNOMA DE
METADADOS: UMA ARQUITETURA PARA CRIAÇÃO E AVALIAÇÃO
DE CONSULTAS SQL GERADAS POR LLMS**

Monografia apresentada à Coordenação de Engenharia de Computação do Campus Timóteo do Centro Federal de Educação Tecnológica de Minas Gerais para obtenção do grau de Bacharel em Engenharia de Computação.

Orientador: Aléssio Miranda Júnior
Coorientador: Evandrino Gomes Barros

Timóteo

2026

Resumo

Este trabalho propõe e avalia um pipeline *Text-to-SQL* orientado por metadados, no qual um agente LLM consulta ferramentas do MCP antes da geração de consultas. O objetivo é reduzir alucinações em tempo de execução e aumentar a executabilidade da SQL em comparação com abordagens baseadas apenas em *schema* estático no *prompt*. O artefato integra Spring AI, servidor MCP e camada semântica e de metadados (catálogo canônico implementado com Apache Atlas), com avaliação experimental em ambiente controlado por métricas de aderência estrutural, executabilidade, rastreabilidade e custo financeiro da operação do pipeline por sessão.

Palavras-chave: *Text-to-SQL*; Model Context Protocol; camada semântica e de metadados; agentes LLM.

Abstract

This work proposes and evaluates a metadata-grounded *Text-to-SQL* pipeline in which a LLM agent consults MCP tools before generating queries. The main goal is to reduce runtime hallucinations and improve SQL executability compared to approaches based only on a static *schema* in the *prompt*. The artifact integrates Spring AI, an MCP server, and a semantic and metadata layer (canonical catalog implemented with Apache Atlas), and is evaluated in a controlled environment using metrics of structural adherence, executability, traceability, and financial cost of pipeline operation per query.

Keywords: *Text-to-SQL*; Model Context Protocol; semantic and metadata layer; LLM agents.

Lista de ilustrações

Figura 1 – Fluxo lógico do artefato alvo.	10
Figura 2 – Hierarquia do paradigma de inteligência artificial.	13
Figura 3 – Arquitetura lógica do MCP.	15
Figura 4 – Fluxos metodológicos: preparação da massa de avaliação e ciclo experimental.	24
Figura 5 – Topologia do artefato: plano de aplicação e plano de dados.	27
Figura 6 – Pacote de evidências por corrida experimental (modo MCP).	33
Figura 7 – Arquitetura de referência do Apache Atlas no ecossistema ODP.	34
Figura 8 – Topologia de rede: zonas lógicas e implantação AWS.	35

Lista de tabelas

Tabela 1 – Catálogo MCP: quinze <i>tools</i> de descoberta sobre o Apache Atlas.	26
Tabela 2 – Indicadores essenciais por modo experimental.	31
Tabela 3 – Bateria de perguntas da avaliação.	32
Tabela 4 – Mecanismos de contexto e decisão.	34
Tabela 5 – Registro da campanha <i>baseline-static</i>	36
Tabela 6 – Cronograma proposto.	37
Tabela 7 – Massa de perguntas da bateria (<i>batteryVersion</i> v1).	39

Lista de abreviaturas e siglas

BERT	<i>Bidirectional Encoder Representations from Transformers</i>
BIRD	<i>Big Bench for Large-Scale Database</i>
ETL	<i>Extract, Transform, Load</i> (extração, transformação e carga)
GPT	<i>Generative Pre-trained Transformer</i>
IA	Inteligência artificial
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i> (modelo de linguagem de grande escala)
MCP	<i>Model Context Protocol</i>
ML	<i>Machine Learning</i> (aprendizado de máquina)
NL	<i>Natural Language</i> (linguagem natural)
NLI-DB	<i>Natural Language Interface to Database</i> (interface em linguagem natural para bases de dados)
PLN	Processamento de linguagem natural
RAG	<i>Retrieval-Augmented Generation</i> (geração aumentada por recuperação)
Seq2Seq	<i>Sequence-to-Sequence</i> (sequência a sequência)
SKG	<i>Structured Knowledge Grounding</i> (ancoragem de conhecimento estruturado)
SQL	<i>Structured Query Language</i> (linguagem estruturada de consulta)
VPC	<i>Virtual Private Cloud</i> (nuvem privada virtual)
IaaS	<i>Infrastructure as a Service</i> (infraestrutura como serviço)

Sumário

1	INTRODUÇÃO	9
1.1	Contexto e motivação	9
1.2	Problema de pesquisa	9
1.3	Justificativa	9
1.4	Objetivos	10
2	FUNDAMENTAÇÃO TEÓRICA	11
2.1	Interfaces em Linguagem Natural e Bases de Dados	11
2.2	Text-to-SQL	12
2.3	Modelos de Linguagem e Mediação Instrumental	13
2.4	Model Context Protocol (MCP)	14
2.5	Vinculação de Esquema (<i>schema linking</i>)	15
2.6	Contextualização de Dados e Ancoragem Semântica	16
2.6.1	Contexto de Domínio e Pré-processamento	16
2.6.2	Grounding e a Camada Semântica e de Metadados	17
2.7	Coerência Lógico-estrutural e de Negócio da Consulta	17
2.8	Síntese	18
3	TRABALHOS RELACIONADOS	20
4	PROCEDIMENTOS METODOLÓGICOS	23
4.1	Ambiente, materiais e dados	23
4.2	Métodos	24
4.2.1	Unidade experimental e vocabulário	25
4.2.2	Arquitetura do artefato	25
4.2.3	Procedimento de Execução	28
4.2.4	Protocolo de Avaliação	28
4.2.5	Indicadores e Agregação	29
4.2.5.1	Métricas do Artefato Proposto	30
4.2.5.2	Métricas da Baseline Comparativa e Infraestrutura	31
4.2.6	Registro Experimental	33
4.3	Escolhas Arquiteturais	33
4.3.1	Vinculação Progressiva em Detrimento de Injeção Estática e RAG	33
4.3.2	Apache Atlas frente a Fontes Paralelas de Esquema	34
4.3.3	Cluster ODP e Massa Real frente a Recortes Simplificados	34
4.3.4	Protocolo do Comparativo Simples para Baseline	36
4.3.5	Execução e Registro da Campanha	36
5	CRONOGRAMA DE EXECUÇÃO	37

	APÊNDICES	38
	APÊNDICE A – BATERIA DE AVALIAÇÃO E GABARITO SQL	39
A.1	Massa de perguntas	39
A.2	Gabarito SQL de referência	40
	REFERÊNCIAS	47

1 Introdução

1.1 Contexto e motivação

Com o avanço de técnicas de *Machine Learning* (ML), modelos LLM têm sido cada vez mais aplicados no contexto de tradução, como no caso de linguagem natural ser traduzida para linguagens formais declarativas e parcialmente sensíveis ao contexto como o SQL. Neste sentido, técnicas de codificação-decodificação *Sequence-to-Sequence* (Seq2Seq) evoluem para aplicarem as técnicas de aprendizado de máquina para otimização das expressões formais criadas a partir de linguagem natural.

1.2 Problema de pesquisa

Em consequência do surgimento de tecnologias de *Deep Learning* e consequente consumo por parte não técnica da população, interfaces de *Text-to-SQL* ganharam notoriedade no cenário de aplicações onde é possível realizar consultas de variada complexidade em bases de dados sem prévio conhecimento de SQL (LIU et al., 2026). Entretanto, apesar do significativo avanço dessa tecnologia, ainda existem gargalos no processo de tradução dessas linguagens de diferentes domínios. Isso posto, as duas principais causas de problemas de tradução, são, primeiramente, a ambiguidade semântica de qualquer linguagem natural e a consequente baixa interpretabilidade dos modelos quando expostos à expressões complexas, como também, as restrições no domínio semântico do SQL, quando da tentativa de replicar em linguagem regular, as operações desejadas especificadas na linguagem de origem.

O problema investigado consiste em como implementar uma arquitetura, que vise aumentar a correlação com a busca de origem em linguagem natural, a aderência estrutural e a executabilidade de consultas SQL geradas por LLMs, mantendo rastreabilidade do processo decisório e executivo. Em específico, busca-se reduzir alucinações estruturais como referência a tabelas, colunas ou relacionamentos inexistentes, assim como, inadequação à intenção de origem, no contexto da consulta e dos dados consultados.

1.3 Justificativa

Este trabalho se justifica pela necessidade de, através da implementação e estudo da tarefa de processamento de linguagem natural (PLN), *Text-to-SQL*, que implementa tecnologia de aprendizado de máquina, tornar a barreira técnica de realização de consultas SQL em bases de dados, por parte não técnica da população, cada vez menor. Consonantemente, impulsionar a utilização desse modelo de tradução, no contexto de melhorar a qualidade e confiabilidade de decisões tomadas conforme insights extraídos a partir de dados.

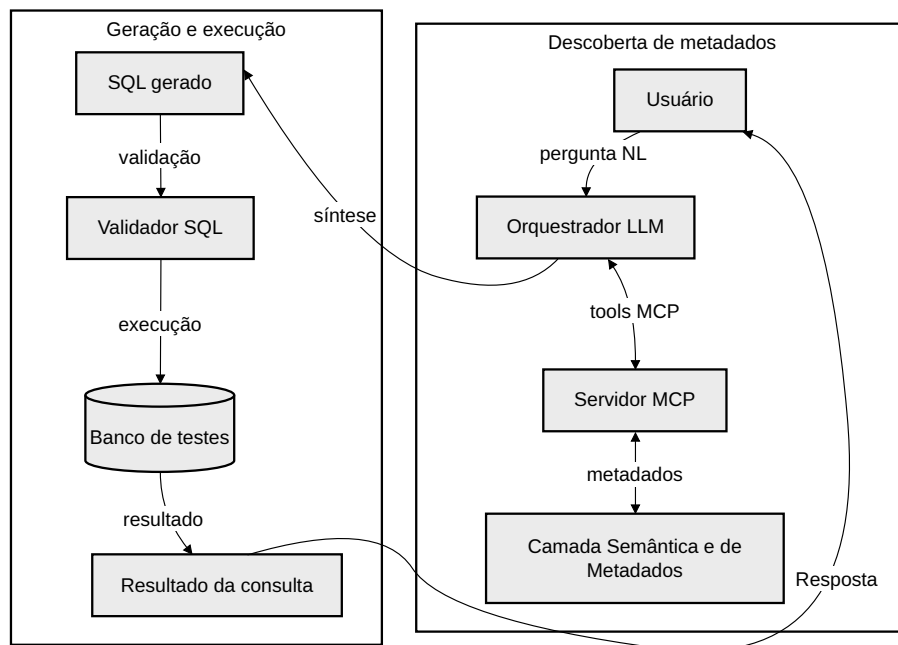
No presente documento, também é possível ver, além de uma proposta arquitetônica da implementação do artefato alvo, também um estudo dirigido em caso de uso, onde é avali-

ado de forma crítica o desempenho do artefato, considerando as métricas de análise de alucinações estruturais e executabilidade de consultas SQL.

1.4 Objetivos

O objetivo de execução do presente projeto, tem como alvo construir e avaliar um artefato *Text-to-SQL* mediado por servidor MCP e apoiado por camada semântica e de metadados vinculada a um banco de dados relacional, com foco na redução de alucinações estruturais e no aumento da confiabilidade da geração de sentenças SQL executáveis conforme ilustrado na Figura 1.

Figura 1 – Fluxo lógico do artefato alvo.



Fonte: Elaboração própria.

São definidos como objetivos específicos deste trabalho, os que se seguem:

1. Projetar a arquitetura do artefato com orquestrador LLM, servidor MCP e adaptador da camada semântica;
2. Definir e implementar *tools* MCP para descoberta semântica da base de dados alvo;
3. Avaliar o artefato em ambiente controlado com métricas de aderência (estrutural, sintaxe, conformidade com a intenção de origem), latência e custo FinOps por sessão (tokens LLM);

2 Fundamentação Teórica

2.1 Interfaces em Linguagem Natural e Bases de Dados

Interfaces em Linguagem Natural para Bases de Dados (NLI-DB) tratam do espaço de desenho em que intenções formuladas em linguagem natural (NL) devem ser mapeadas para operações consultivas sobre repositórios estruturados (LIU et al., 2026). Do ponto de vista conceitual, há uma assimetria fundamental: a NL é tolerante a ambiguidade e omissão, enquanto o modelo relacional exige predicados explícitos sobre um esquema declarado e um vocabulário de símbolos estáveis. Neste contexto, assumimos essa mediação como problema central de representação, anteriormente a quaisquer definições e escolhas arquiteturais ou metodológicas.

Segundo (ANDROUTSOPOULOS; RITCHIE; THANISCH, 1995), no final da década de 1960 já existiam protótipos que propunham interface em linguagem natural para bases de dados (NLI-DB) para a interface de bases de dados em NL. Nessa época, o modelo mais conhecido proposto foi LUNAR que era usado para análise de composição química em pedras lunares. Assim como para outras propostas, mesmo que houvesse modelagem interna para generalizar o domínio semântico da aplicação, o uso ainda era muito vinculado ao domínio de negócio dos dados persistidos.

Nas décadas seguintes do século XX, modelos propunham contextos mais avançados, como estabelecimento de perguntas ao usuário antes da criação da consulta, intercalação de processamento sintático e semântico, transformação de NL em expressões lógicas de linguagens declarativas, como já foi feito com Prolog e ensinar o sistema novas palavras durante a interação, assim como fazia o Ask, outro sistema de mesma classe.

Neste contexto, em função da evolução do paradigma, modelos metodológicos foram propostos com o objetivo de resolver o mesmo problema. Uma síntese recente (LIU et al., 2026) permite organizá-los da seguinte forma:

1. **Regras:** Árvore de análise sintática, ontologia, grafo semântico.
2. **Aprendizado de máquina:** Redes neurais, aprendizado de máquina tradicional.
3. **Modelos de linguagem pré-treinados:** *Bidirectional Encoder Representations for Transformers* (BERT)-based, modelos baseados em grafo, *template filling*, métodos generativos puros.
4. **LLM:** engenharia de *prompt*, *fine-tuning*, estrutura integrada de conhecimento de base de dados.

Dessa forma, os modelos de interface entre NL e linguagem formal evoluíram de forma a aplicar conceitos de *Seq2Seq* (SUTSKEVER; VINYALS; LE, 2014) para realizar a geração

direta das expressões formais criadas a partir de NL.

A arquitetura Seq2Seq generalizou a ideia de mapear um enunciado em NL para um programa simbólico de comprimento variável, abrindo caminho a modelos neurais de análise semântica em que a atenção alinha fragmentos da pergunta aos constituintes da linguagem alvo (BAHDANAU; CHO; BENGIO, 2014). Com o predomínio de representações contextualizadas pré-treinadas e, mais recentemente, de modelos generativos de grande escala, o mesmo enquadramento passou a ser explorado sob pressões de generalização a esquemas e consultas inéditos, consolidando a tarefa de *Text-to-SQL* como caso paradigmático de interface consultiva (YU et al., 2018; LIU et al., 2026). O que se segue caracteriza essa tarefa e as exigências de *schema linking* e de composição sintática de *SQL* que dela decorrem.

2.2 Text-to-SQL

Text-to-SQL nomeia a tarefa de produzir uma consulta em *SQL*, semanticamente compatível com um esquema disponível, a partir de uma pergunta em NL (LIU et al., 2026). Ela herda a tensão referida acima: é preciso resolver ambiguidades da pergunta, relacionar termos do usuário a nomes de relações e atributos (*schema linking*) e compor uma formulação executável no dialeto pretendido. Benchmarks clássicos formulam a tarefa em regimes onde esquemas ou consultas não reaparecem integralmente entre treino e teste, explicitando a exigência de generalização (YU et al., 2018).

As configurações posteriores de grande escala continuam a evidenciar dependência entre pergunta, evidência de esquema ou dados e o comportamento observado de sistemas centrados em modelos de linguagem (LI et al., 2023). A tarefa de *Text-to-SQL* evoluiu de uma forma controlada, com técnicas baseadas em regras, para uma forma flexível, com paradigma centralizado em NL. Essa mudança leva a *trade-offs* entre acurácia, interpretabilidade, generalização, eficiência de custo e transferibilidade de domínio. Para reduzir a lacuna entre os estudos acadêmicos e as aplicações do mundo real, propostas mais modernas como o *benchmark Big Bench for Large-Scale Database* (BIRD) (LI et al., 2023) realocaram o foco da mera compreensão estrutural do esquema para a complexidade dos valores reais armazenados. Esse novo formato evidencia que ainda há desafios que permanecem em aberto, notadamente: a dificuldade de lidar com bases de dados massivas que contêm valores ruidosos; a necessidade de raciocínio baseado em conhecimento externo para alinhar a intenção do usuário aos dados de domínio; e a exigência por eficiência de execução (*execution efficiency*) das consultas geradas.

Conforme demonstrado pelas avaliações do BIRD, a acurácia dos modelos disponíveis em 2023 (como o *Generative Pre-trained Transformer* (GPT)-4) ainda ficava muito aquém da performance humana (cerca de 54,89% contra 92,96%), mostrando que a compreensão profunda dos valores do banco de dados para a geração de consultas semanticamente e logicamente corretas continuava a ser um fator de desenvolvimento (LI et al., 2023).

O método DIN-SQL (POURREZA; RAFIEI, 2023) exemplifica uma resposta metodológica a essa lacuna: decompor a geração em subproblemas sequenciais, reinserir as soluções

intermediárias no contexto (*decomposed in-context learning*) e acrescentar etapa de autocorreção assistida pelo próprio LLM, mitigando a dificuldade de gerar SQL declarativo de uma só vez a partir da pergunta. Em experimentos reportados pelos autores, o ganho face a *few-shot* simples situa-se da ordem de dez pontos percentuais nos modelos considerados; no conjunto de teste *holdout* do Spider, com GPT-4, alcança-se 85,3% de *execution accuracy*, superando o melhor resultado então citado para a mesma métrica (79,9%); no BIRD, obtém-se 55,9% de precisão de execução no *holdout*, então apresentado como estado da arte nesse *benchmark* de 2023. Essa combinação de decomposição, evidência incremental e revisão antecipa a linha de trabalhos subsequentes que revisitam Spider e BIRD com arquiteturas centradas em LLM, como se verifica a seguir.

Em contexto ainda mais atual, trabalhos como (WANG et al., 2025; JIAN et al., 2026) retomam os mesmos benchmarks já estabelecidos, Spider e BIRD, com modelos LLM mais recentes. Nesses estudos, arquiteturas de *Text-to-SQL* centradas em LLM alcançam, em conjunto, faixas reportadas de acurácia de execução da ordem de 85%–88% no conjunto de teste do Spider e de cerca de 65%–77% no BIRD, conforme métricas e partições de avaliação definidas pelos autores.

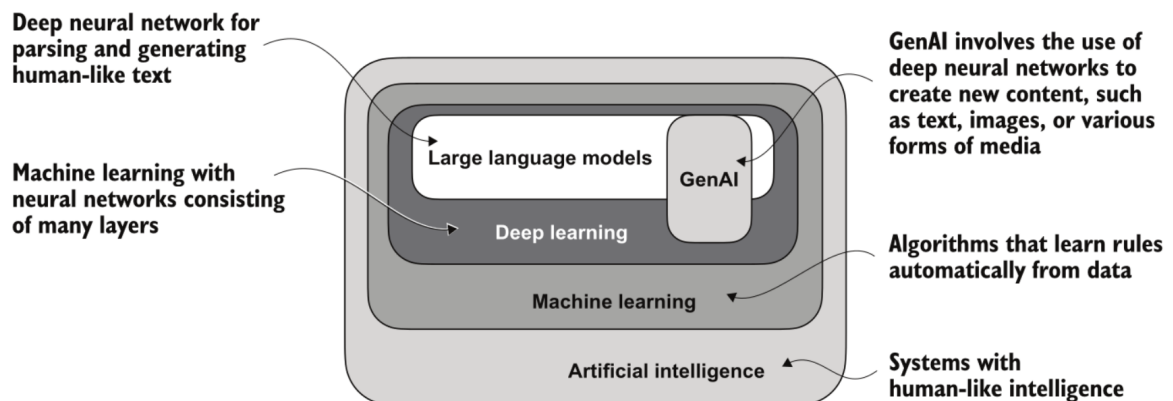
2.3 Modelos de Linguagem e Mediação Instrumental

Os LLMs são, em síntese operacional, modelos de rede neural profunda, treinados em grandes corpus textuais e estruturados em torno da arquitetura *transformer*, capazes de processar e gerar sequências que parecem coerentes e contextualmente adequadas, sem que tal “adequação” implique consciência ou compreensão humanas (RASCHKA, 2024). No âmbito de interfaces entre NL e bases de dados, trabalhos de enquadramento recente posicionam os LLMs como peça central para aproximar intenções em linguagem natural de programas consultivos em *SQL*, prolongando a mediação já discutida para NLI-DB e para a tarefa de *Text-to-SQL*. Nesse papel de camada de tradução, explora-se a geração condicionada ao contexto observado para produzir formulação simbólica executável sob restrições de esquema, motivando o tratamento que se segue.

A Figura 2 demonstra a hierarquia simplificada de inteligência artificial, aprendizado de máquina e *deep learning*, de forma a ilustrar que os LLMs representam uma aplicação específica das técnicas de aprendizado profundo, tal qual que todos estes fazem parte de um paradigma maior de inteligência artificial (IA). Do ponto de vista arquitetural, a proposta clássica do *transformer* descreve pilhas de codificador e de decodificador interligadas por mecanismos de atenção (VASWANI et al., 2017), ponto de partida para situar como a transdução em sequência sustenta a camada de tradução entre NL e consultas em *SQL* já mencionada.

A arquitetura *transformer* introduzida por (VASWANI et al., 2017) segue o esquema codificador–decodificador usual em modelos neurais de transdução: o codificador mapeia uma sequência de símbolos de entrada para uma sequência de representações contínuas intermediárias; dada essa sequência, o decodificador gera a sequência de saída um elemento de cada vez, de modo autorregressivo, e em cada passo o modelo condiciona-se aos símbolos de saída já emitidos. Tanto o codificador como o decodificador são pilhas de camadas idênticas repe-

Figura 2 – Hierarquia do paradigma de inteligência artificial.



Fonte: Adaptado de Raschka (2024).

tidas: em cada camada do codificador combinam-se atenção *multi-head self-attention* e uma rede *feed-forward* pontual, com ligações residuais e normalização de camada; em cada camada do decodificador acrescentam-se, além da *self-attention* mascarada e da mesma rede pontual, um módulo de atenção *multi-head* sobre a saída do codificador. O artigo motiva o desenho com tarefas como a tradução entre línguas; a mesma noção de transdução em sequência ilustra, por analogia, o mapeamento de uma pergunta em NL para um programa em SQL, sem que o artigo original trate explicitamente de bases de dados.

Convém notar que o *Transformer* original é *encoder-decoder*, ao passo que muitos LLM usados hoje em geração de texto e de código adotam variantes *decoder-only* (linha de modelos do tipo GPT e arquiteturas análogas), isto é, uma pilha causal única, sem pilha de codificador separada como na definição acima (RASCHKA, 2024). Mantém-se, contudo, o núcleo de atenção e a geração token a token condicionada ao contexto observado.

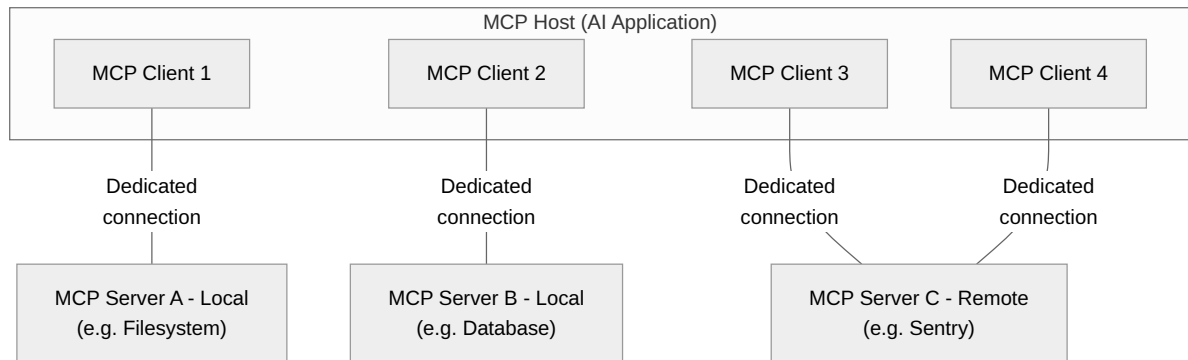
LLMs funcionam, nesse enquadramento de atenção e geração autoregressiva, como mecanismos probabilísticos de geração textual condicionada ao contexto observado. Historicamente, arquiteturas sequenciais com atenção posicionaram-se como referência para alinhar fragmentos da entrada aos símbolos emitidos na saída formal (BAHDANAU; CHO; BENGIO, 2014); em modelos atuais admite-se raciocínio parcelado e recurso a funções externas sempre que o contexto em *prompt* é insuficiente para fixar o programa (LIU et al., 2026).

Em *Text-to-SQL*, esse recurso toma a forma de mediação instrumental: o modelo não depende exclusivamente de uma descrição estática do esquema e pode solicitar informação estruturada em passos sucessivos. A mediação instrumental, por si só, não substitui a necessidade de sincronizar a geração com alguma modelagem que sirva como camada semântica de negócio, tema que as seções seguintes desenvolvem quanto ao protocolo de integração, à vinculação de esquema e ao *grounding* na fonte de metadados.

2.4 Model Context Protocol (MCP)

O MCP é um padrão aberto de integração em que uma aplicação de IA, o *MCP Host*, acessa sistemas externos por meio de servidores que expõem, de forma padronizada, *tools* (ações executáveis), *resources* (dados de contexto) e *prompts* (modelos de interação reutilizáveis) (ANTHROPIC; Model Context Protocol contributors, 2024). O protocolo especifica a troca de contexto entre participantes, e não define como o modelo de linguagem consome esse contexto nem como o agente orquestra o raciocínio.

Figura 3 – Arquitetura lógica do MCP.



Fonte: Adaptado de Anthropic e Model Context Protocol contributors (2024).

Na Figura 3, o *Host* concentra a aplicação que coordena o modelo, de tal forma que para cada *MCP Server* necessário, o *Host* instancia um *MCP Client* que mantém conexão dedicada com esse servidor. Assim, fontes distintas, por exemplo, sistema de arquivos, base de dados ou serviço remoto, integram-se sem acoplar o agente à API interna de cada provedor.

No contexto desta monografia, esse canal tipado é o meio pelo qual o orquestrador *Text-to-SQL* consulta metadados estruturados antes da síntese de *SQL*, mediante *tools* expostas por um servidor MCP que encapsulam o acesso ao catálogo canônico da camada semântica e de metadados. Isso posto, no desenho desta monografia, o *schema linking* progressivo materializa-se como consulta dirigida ao catálogo canônico mediante *tools* MCP, reunindo em passos sucessivos evidência técnica e de negócio sobre o esquema governado.

2.5 Vinculação de Esquema (*schema linking*)

Em *Text-to-SQL*, *schema linking* designa o mapeamento entre a pergunta em NL e os elementos do esquema que a consulta deve empregar: relações, atributos e, quando aplicável, caminhos de junção entre elas (MOHAMMADJAFARI; MAIDA; GOTTUMUKKALA, 2025). O conceito antecede o uso de LLMs e permanece central quando o modelo passa a gerar *SQL* condicionado a um vocabulário relacional fixo: sem vinculação adequada, a saída pode ser sintaticamente correta e entretanto referenciar artefatos inexistentes ou junções incompatíveis com o esquema em manipulação.

Sob esquemas de teste do benchmark Spider, a literatura passa a nomear explicitamente o *schema linking* como o alinhamento entre referências da pergunta em NL e as colunas

ou tabelas pretendidas no esquema (WANG et al., 2019). Nesse cenário cross-domain, RAT-SQL ilustra ambiguidades de referência e a dependência de relações estruturais do banco, inclusive para compor junções válidas entre relações. O IRNet, também avaliado em Spider, formaliza a etapa como primeira fase de um pipeline desacoplado: reconhecer tabelas e colunas mencionadas e tipar colunas conforme o enunciado, mitigando o problema léxico em vocabulário fora do domínio de treino (GUO et al., 2019).

Com LLMs, a seleção de subconjuntos relevantes do esquema permanece distinta da geração de SQL. No benchmark BIRD, que expõe catálogos maiores e descrições de coluna, o framework CHESS emprega um agente de seleção de esquema para reduzir tabelas e colunas antes da síntese, e o estudo considera essa etapa necessária quando o inventário excede o que cabe de forma confiável no contexto do modelo (TALAEI et al., 2024). Essa linhagem reforça que a vinculação não é acessório histórico, mas condição para ancorar a consulta ao esquema efetivamente disponível.

2.6 Contextualização de Dados e Ancoragem Semântica

O avanço da aplicação de recursos de aprendizado de máquina a *pipelines* de tradução entre NL e SQL, embora substancial, ainda expõe dificuldades de pré-processamento de requisitos, seja do ponto de vista do *schema*, dos dados ou do *input* recebido. Isso posto, parte disso advém das dificuldades quando do entendimento do contexto do domínio dos dados consultados, tal qual, no *schema linking* é necessário que haja correlação linguística mínima entre o que é a representação técnica de uma fonte e sua correspondência com o domínio e a pergunta em NL original.

O que se segue organiza esse tema em duas subsecções: primeiro o *contexto de dados* e as etapas de pré-processamento (como referência, *Extract, Transform, Load* (ETL)), e em seguida, *grounding* e a camada semântica e de metadados, onde a literatura demonstra os ganhos de precisão após evidências de esquema estarem disponíveis ao modelo.

2.6.1 Contexto de Domínio e Pré-processamento

Aqui, **contexto de dados** nomeia, de maneira operacional, o conjunto de informações de domínio, de esquema relacional disponível e de entradas textuais que circulam no *pipeline* antes e durante a geração de SQL, ou seja, tudo o que o sistema pode inspecionar para reduzir ambiguidades entre a intenção em linguagem natural e os símbolos do banco.

Pré-processamento reúne etapas como saneamento, harmonização de formatos, duplicação parcial, enquadramento de valores ruidosos ao *schema* e preparação de evidências tabulares ou textuais consumidas por modelos ou por serviços auxiliares. Em conjunto, buscam diminuir ruído e desalinhamentos entre a fonte bruta e o uso analítico ou generativo.

Um referencial clássico de orquestração desses fluxos é o *pipeline* ETL (extração a partir das fontes, transformação segundo regras de negócio e de qualidade, carga em destino analítico ou transacional). Em desenhos com característica de *Data-centric Machine Learning* (ML) Pipeline, o tratamento da qualidade dos dados e a definição de artefatos reutilizáveis no

treino ou na inferência situam-se nessa cadência, de modo que, antes de se pleitear ganho de acurácia na sentença final, estabiliza-se o que será observado pelo modelo. Essa ênfase em engenheirar os dados antes de buscar ganhos no modelo alinha-se ao paradigma *data-centric*, no qual a qualidade e a manutenção dos dados ao longo do seu ciclo de vida, e não apenas o ajuste do modelo, sustentam o desempenho final (ZHA et al., 2025).

Metadados descritivos e de governança, como inventário de relações e atributos, tipos, *constraints* e linhagem, são, em geral, *registrados após* (ou em continuidade explícita com) etapas de integração e de padronização dos dados. Uma vez que objetos passíveis de descrição estável resultam da extração, da transformação e da carga, o catálogo pode formalizar declarações auditáveis sobre o que “existe” no sistema de informação, em complemento ao volume físico de tuplas. Referenciais de gestão de dados tratam metadados e catálogos como disciplina de governança que descreve os ativos de dados da organização (DAMA International, 2017), o que explica por que o enriquecimento por metadados aparece, na prática, como camada posterior ao pré-processamento, e não como substituto dele. Estabilizado o que entra no *pipeline*, abre-se espaço para ancorar a geração às evidências de esquema, tema da subseção seguinte.

2.6.2 Grounding e a Camada Semântica e de Metadados

No contexto da tentativa de aumentar a acurácia da sentença *SQL* final por meio da aplicação de catálogos de metadados, relatos na literatura alcançam, por exemplo, 91,8% de precisão (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025).

Grounding, no presente uso, designa a disciplina de amarrar decisões de geração a evidências observáveis sobre o domínio dos dados. Essa noção conecta-se ao eixo de *Structured Knowledge Grounding* (SKG), em que pedidos formulados em linguagem natural são ancorados em conhecimento estruturado, como bancos de dados, para produzir programas como o *SQL* (XIE et al., 2022). Metadados de catálogo, como nomes, tipos, *constraints*, descrições textuais e relacionamentos, funcionam como uma camada de declarações auditáveis sobre o que “existe” no sistema de informação, distinta do volume de tuplas armazenadas. Quando preservados e consultados de forma explícita, contribuem para reduzir o espaço de hipóteses compatíveis sobre tabelas e caminhos de junção.

Estudos de geração de *Text-to-SQL* com enriquecimento explícito por metadados ilustram esse princípio (WANG et al., 2025), e trabalhos sobre geração guiada por catálogo em pipelines centrados em dados, domínio próximo ao que este trabalho explora, reforçam o papel da governança de metadados como sinal de orientação para modelos generativos (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025). Com o vocabulário relacional e as declarações de catálogo fixados como evidência consultável, o problema desloca-se para verificar se a consulta gerada permanece dentro desse espaço estrutural, tema da seção seguinte.

2.7 Coerência Lógico-estrutural e de Negócio da Consulta

A confiabilidade da síntese distingue, para os efeitos desta monografia, a **coerência estrutural**, em que a consulta referencia apenas artefatos presentes no esquema sob governança, de noções mais amplas de correção de negócio, ligadas à adequação do resultado à intenção analítica original do usuário. O escopo deste trabalho concentra-se na primeira, por ser verificável contra a camada semântica e de metadados já delimitada, enquanto a segunda depende de critérios de domínio que extrapolam o vocabulário relacional disponível.

Recebendo o fechamento da subseção anterior, em que o catálogo delimita o espaço estrutural de hipóteses, os erros estruturais tornam-se verificáveis contra a camada de metadados. Inventar uma relação ou um atributo inexistente, ou compor uma junção sem suporte nos relacionamentos declarados, contraria diretamente a camada de metadados, independentemente de o resultado parecer “plausível” quando lido em NL. Essa propriedade separa o desvio estrutural de uma resposta apenas inconveniente, pois o primeiro pode ser confrontado com as declarações de catálogo, ao passo que a plausibilidade superficial em linguagem natural não oferece, por si, garantia de aderência ao esquema, visto que pode tentar correlacionar com dados não existentes na fonte.

A literatura recente sobre detecção de alucinações em *Text-to-SQL* assistido por LLM reforça essa leitura. O trabalho de (YANG et al., 2025) caracteriza esses desvios em classes distintas, entre elas a alucinação de esquema, quando o modelo invoca tabelas ou colunas inexistentes, e a alucinação lógica, quando as cláusulas geradas contradizem a intenção expressa na pergunta. Para identificar tais falhas sem depender de respostas gabarito, os autores propõem um procedimento de teste metamórfico em duas fases, uma voltada à vinculação de esquema e outra à síntese lógica, no qual perturbações controladas na entrada e o cruzamento das saídas correspondentes revelam inconsistências. O relevante para esta fundamentação é que a verificação de coerência estrutural pode prescindir de um resultado esperado que sirva como referência, apoiando-se na confrontação com a estrutura declarada, isso em favor do prévio conhecimento quanto ao domínio dos dados.

Essa diferenciação orienta o desenvolvimento metodológico do presente trabalho, que adota uma definição operacional de **erro estrutural** ancorada no catálogo canônico, detalhada no capítulo de metodologia, e a emprega como critério de validação e de classificação de desfecho. Com isso, a confiabilidade da síntese passa a ser medida pela aderência ao esquema governado e pela trilha do protocolo de integração que dá acesso a essa evidência, conforme a proposta arquitetônica e procedimental.

2.8 Síntese

O percurso deste capítulo parte do problema de mediar intenções em linguagem natural para consultas formais sobre dados estruturados, no qual a tolerância da NL à ambiguidade e à omissão contrasta com a exigência de predicados explícitos do modelo relacional. A tarefa de *Text-to-SQL* concentra essa tensão, pois requer resolver ambiguidades da pergunta, relacionar termos do usuário a relações e atributos por meio de *schema linking* e compor uma

consulta executável sobre um vocabulário relacional fixo.

Sobre esse problema atuam os modelos de linguagem como camada de tradução, capazes de gerar formulações simbólicas condicionadas ao contexto observado e de recorrer à mediação instrumental quando a descrição estática do esquema não basta. O MCP formaliza esse acesso ao expor de modo padronizado a consulta progressiva ao catálogo, enquanto o *grounding* na camada semântica e de metadados ancora as decisões de geração em declarações auditáveis sobre o que existe no sistema de informação. A montante, etapas de pré-processamento e de integração estabilizam o que será observado, de modo que o enriquecimento por metadados se apresente como camada posterior de governança, e não como substituto do pré-processamento, conforme ilustrado no capítulo de metodologia.

Desse arranjo decorre o critério de confiabilidade adotado, centrado na coerência estrutural da consulta, isto é, na referência apenas a artefatos presentes no esquema sob governança, distinta de noções mais amplas de correção de negócio. Erros estruturais são verificáveis pela camada de metadados, o que permite avaliá-los sem depender exclusivamente de um resultado tabular de referência.

O capítulo seguinte situa o estado da arte e os trabalhos que instanciam essas dimensões com abordagens nomeadas e comparáveis, organizados em torno do uso de metadados no *schema linking*, da exploração progressiva do esquema por ferramentas e da robustez e validação da geração assistida por LLM.

3 Trabalhos Relacionados

Existem três eixos que situam o estado da arte, o uso de metadados no *schema linking*, a exploração progressiva do esquema por chamadas de ferramenta e a robustez e validação da geração assistida por LLM. Para este capítulo, adota-se aqui o critério de selecionar obras que instanciam cada um desses eixos com abordagem nomeada e comparável à linha conceitual do presente trabalho, a saber, o apoio em catálogo de metadados, a consulta dirigida ao esquema e a trilha observável da cadeia decisória. O panorama recente da tradução de linguagem natural para SQL é tomado a partir do mapa taxonômico organizado por (MOHAMMADJAFARI; MAIDA; GOTTUMUKKALA, 2025).

No primeiro eixo, o trabalho de (WANG et al., 2025) parte da constatação de que muitos erros de geração concentram-se na etapa de *schema linking*, quando entidades da pergunta não são corretamente associadas a tabelas e colunas do banco. Para reduzir essa falha, os autores atribuem a cada coluna um contexto de metadados completo durante a vinculação e introduzem um mecanismo de correção intermediária que valida e ajusta a consulta ainda no processo de geração, em vez de avaliar apenas o resultado final. Segundo os autores, essa estratégia alcança acurácia de execução de 74,45% no conjunto de desenvolvimento e 76,41% no conjunto de teste do *benchmark* BIRD (LI et al., 2023). A leitura desse resultado reforça, para o presente trabalho, o papel dos metadados de catálogo como evidência que orienta a geração, ainda que aqui o acesso a essa evidência se dê por consulta dirigida ao catálogo, e não por injeção do contexto montado a montante no *prompt*, conforme o capítulo seguinte detalha.

Ainda no eixo anterior, (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025) aborda a geração de *pipelines* centrados em informação com o apoio de LLM e atribui papel central a um catálogo de dados, do qual extrai metadados, deriva instruções específicas por conjunto de dados e conduz um ciclo iterativo de geração e correção das saídas. Essa linha de governança por catálogo orienta a continuidade adotada neste trabalho, cujas escolhas de materialização da camada semântica e de metadados ficam reservadas às escolhas arquiteturais registradas no próximo capítulo. Convém distinguir, contudo, o escopo, pois o presente trabalho restringe-se a um objeto *Text-to-SQL* único, sobre massa relacional fixa e governada, e não na construção de um *pipeline* de aprendizado de máquina completo como o discutido pelos autores.

O segundo eixo reúne abordagens que tratam o acesso ao esquema como exploração ativa. Em (JIAN et al., 2026), o cenário de *unknown schema* é formalizado como processo de decisão parcialmente observável, no qual o agente descobre tabelas, colunas e relacionamentos por *tool calls* em múltiplos turnos, com a política de interação refinada por aprendizado por reforço, característica do estudo e não compromisso assumido neste trabalho. De forma complementar, (TALAEI et al., 2024) propõe seleção contextual do esquema quando o inventário excede o que cabe de forma confiável no contexto do modelo, problema já referido no

capítulo de fundamentação. Essa literatura de exploração por ferramentas e de seleção de esquema orienta o eixo de vinculação progressiva perseguido aqui, cuja operacionalização é remetida ao capítulo seguinte. Contrasta com ela a injeção estática do esquema no *prompt*, como em (POURREZA; RAFIEI, 2023), que oferece menor margem para uma trilha observável das decisões de acesso.

Nessa linha multi-agente, (WANG et al., 2025) propõe MAC-SQL, com três papéis colaborativos: um agente Selector filtra sub-esquema quando o inventário excede o limite de contexto, um Decomposer decompõe a pergunta e um Refiner recorre à execução em SQLite para corrigir SQL incorreto. Segundo os autores, a combinação com GPT-4 alcança 59,59% de acurácia de execução no conjunto de teste *holdout* do BIRD (LI et al., 2023). A leitura desse framework reforça a exploração por ferramentas e a decomposição da tarefa, mas o esquema permanece montado a montante no *prompt*, sem consulta dirigida ao catálogo canônico governado nem trilha padronizada das decisões de acesso comparável à vinculação progressiva perseguida neste trabalho.

O terceiro eixo volta-se à robustez e à validação da geração. O método SQLHD, proposto por (YANG et al., 2025), detecta alucinações em *Text-to-SQL* assistido por LLM mediante testes metamórficos em duas fases, uma voltada à vinculação de esquema e outra à síntese lógica, sem depender de um gabarito tabular para cada consulta. Segundo os autores, a abordagem atinge F1-score entre 69,36% e 82,76% conforme os cenários avaliados, superando estratégias de autoavaliação pelo próprio modelo. Essa distinção entre desvio de esquema e desvio de lógica dialoga diretamente com o critério de coerência estrutural adotado na fundamentação deste trabalho, e o método é tomado aqui como referência metodológica para a validação, e não como objeto a ser reimplementado de forma integral. Cabe ainda registrar uma limitação pertinente, pois a execução bem-sucedida de uma consulta não garante a correção quanto à intenção de negócio, de modo que o escopo aqui considerado permanece o erro estrutural verificável contra o catálogo, num protocolo controlado de corridas de verificação.

Complementarmente, (CHATURVEDI; CHADHA; BINDSCHAEDLER, 2025) propõe SQL-of-Thought, pipeline multi-agente que combina *schema linking*, identificação de subproblemas, plano de consulta e loop de correção guiada por taxonomia de erros, o que inclui falhas de vinculação e de junção. Segundo os autores, a abordagem atinge 91,59% de acurácia de execução no Spider 1.0 no conjunto de desenvolvimento. Essa linha dialoga com SQLHD ao distinguir desvios de esquema e de lógica, mas a avaliação holística permanece centrada em resultados tabulares da execução, não em aderência estrutural verificável contra catálogo governado, no mesmo limite já registrado acima quanto à intenção de negócio.

Tomados em conjunto, e à luz do mapa de (MOHAMMADJAFARI; MAIDA; GOTTUMUKKALA, 2025) e da revisão complementar de (HONG et al., 2025), os trabalhos distribuem-se pelos três eixos de forma geral. As propostas centradas em metadados, como (WANG et al., 2025) e (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025), robustecem o *grounding* estrutural ao tratar o catálogo como fonte de evidência para a geração. As abordagens de exploração, como (JIAN et al., 2026), (TALAEI et al., 2024) e (WANG et al., 2025), qualificam o mecanismo de acesso ao esquema por chamadas de ferramenta ou filtragem progressiva. Os métodos de

(YANG et al., 2025) e de (CHATURVEDI; CHADHA; BINDSCHAEDLER, 2025), por sua vez, fortalecem a verificação e a correção posterior da saída.

Sobre o plano de avaliação, (LEI et al., 2025) introduz o Spider 2.0 como evolução dos conjuntos clássicos (YU et al., 2018; LI et al., 2023) para workflows empresariais de *Text-to-SQL*, com esquemas de centenas de colunas e tarefas que exigem busca de metadados, documentação de dialetos e, no *setting agêntico*, interação iterativa mediante o *Spider-Agent* proposto pelos autores. Segundo os autores, desempenho elevado nos benchmarks tradicionais (91,2% no Spider 1.0 e 73,0% no BIRD) contrasta com 21,3% de taxa de sucesso no Spider 2.0 e 5,68% de acurácia de execução no Spider 2.0-lite, o que reforça que a dificuldade persiste sobretudo no *schema linking* em escala (cerca de 27,6% dos erros analisados). Convém distinguir o papel desse benchmark: mede sucesso por execução em cenários empresariais complexos, e o contraste com os percentuais elevados nos conjuntos clássicos evidencia que volumetria e complexidade de esquema real permanecem desafiadoras para LLMs. Isso posto, não propõe catálogo canônico de governança nem avaliação de coerência estrutural contra catálogo governado, distinta da aderência estrutural verificável perseguida neste trabalho.

Dessa leitura emergem lacunas que o presente trabalho busca endereçar, a saber, (i) a escassez de protocolos experimentais reproduzíveis que registrem a orquestração decisória do agente (pergunta, consultas ao catálogo e SQL) num artefato laboratorial único, (ii) a falta de comparação quantitativa entre mediação via protocolo padronizado (MCP) sobre catálogo canônico e injeção estática de esquema no *prompt*, e (iii) o fato de benchmarks empresariais recentes ampliarem escala e complexidade, mas avaliarem sobretudo execução, sem verificar coerência estrutural contra catálogo governado. Diante disso, posiciona-se este trabalho como artefato *Text-to-SQL* em que a consulta à camada semântica e de metadados, mediada por MCP, antecede a geração de SQL, com avaliação por aderência estrutural, executabilidade, rastreabilidade e custo de interação. O protocolo experimental que operacionaliza essa resposta, incluindo o comparativo simples por injeção estática de esquema, que sustenta a comparação quantitativa da lacuna (ii), e é detalhado no Capítulo de Metodologia.

4 Procedimentos Metodológicos

A presente pesquisa possui natureza aplicada, exploratória e experimental no contexto dos objetivos definidos no Capítulo de Introdução. Adota-se como objeto de estudo um único artefato *Text-to-SQL* mediado por MCP, no qual a descoberta de metadados antecede a geração de SQL. O interesse metodológico central reside na avaliação sistemática da aderência estrutural, da executabilidade e da rastreabilidade da cadeia decisória do artefato em ambiente laboratorial reproduzível (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025; LI et al., 2023).

4.1 Ambiente, materiais e dados

O ambiente metodológico da pesquisa é organizado por papéis funcionais e fluxos lógicos do objetivado artefato de tarefa *Text-to-SQL*. Para garantir rastreabilidade do experimento, os materiais são agrupados por função no ciclo de execução.

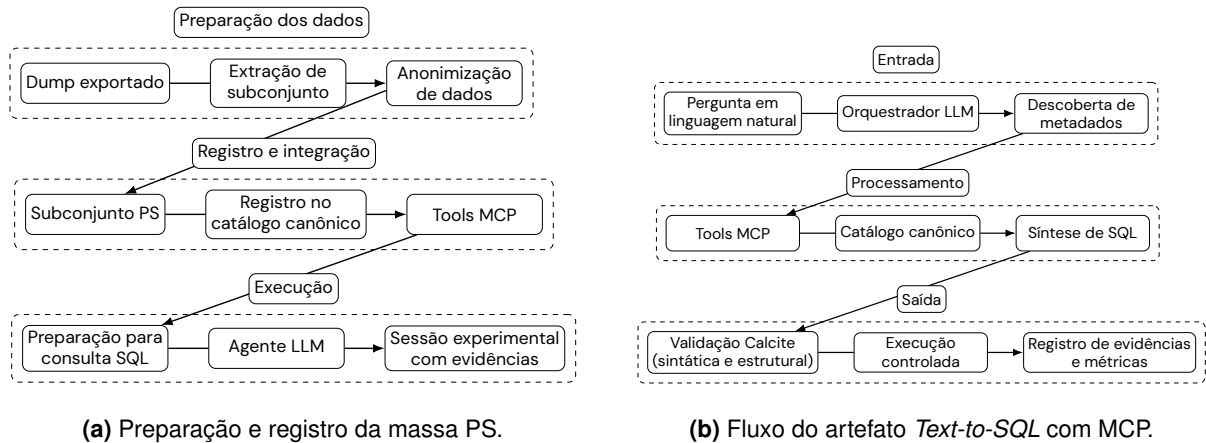
Conforme a Fundamentação Teórica, a **camada semântica e de metadados** expõe ao agente inventário estruturado de esquema e governança via MCP. Neste capítulo, essa camada materializa-se experimentalmente como **catálogo canônico de metadados**, fonte única de verdade sobre o subconjunto de esquema governado na corrida. No laboratório, esse catálogo é implementado pelo **Apache Atlas**, que registra as entidades técnicas do subconjunto PS (92 tabelas), após a carga em Hive e as expõe às *tools* MCP de descoberta. A justificativa da escolha do Atlas frente a fontes paralelas de esquema é detalhada na seção de escolhas arquiteturais.

- **Software:** Java 25, Spring Boot 4.1.0, Spring AI e servidor MCP.
- **Camada semântica e dados:** catálogo canônico de metadados implementado em Apache Atlas e massa relacional PS (92 tabelas; subconjunto fixo para avaliação).
- **Execução e evidências:** validador sintático Apache Calcite (dialetto Hive), executor SQL controlado em Hive e *harness* experimental para trilha de sessão.
- **Modelos de linguagem:** OpenAI *gpt-5.4-nano* e Google *gemini-3.5-flash*, com *modelVersion* fixada por corrida.
- **Infraestrutura de computação:** cinco instâncias EC2 em AWS (*x86_64*, Ubuntu 24.04):
 - **Serviço Web Spring (1):** (8 vCPU, 32 GiB RAM, EBS gp3) na zona *NetInternalApp*, separada do cluster de dados (Figura 8).
 - **Cluster ODP (4):** um *master* *m6i.2xlarge* (8 vCPU, 32 GiB, EBS gp3 $\geq$ 100 GiB) com Ambari, ZooKeeper, NameNode, ResourceManager e Apache Atlas; três *workers* *m6i.2xlarge* (8 vCPU, 32 GiB cada, EBS gp3 $\geq$ 100 GiB por nó) para HDFS, YARN, Hive, HBase e Kafka (ODP 1.3.1.0).

- **Ambiente laboratorial:** arquitetura em IaaS AWS; a topologia lógica do artefato é explicada adiante na Figura 5.

A preparação da massa prioriza encadeamento auditável desde a extração até a disponibilização para sessões experimentais, conforme o painel (a) da Figura 4 ao final deste capítulo.

Figura 4 – Fluxos metodológicos: preparação da massa de avaliação e ciclo experimental.



Fonte: Elaboração própria.

No plano de dados, adota-se como fonte relacional o domínio PS (92 tabelas), com subconjunto estável de tabelas representativas dos cenários de avaliação (franquias, projetos, transações, produtos, publicações entre outros domínios). A massa de avaliação consiste em subconjunto anonimizado derivado da base operacional da empresa, também denominada Putz Studio. Esse recorte permite executar consultas simples, junções, filtros temporais e agregações em um corpus coerente com o problema investigado. O dicionário físico completo permanece como referência documental, enquanto o subconjunto efetivamente exposto ao agente é governado pelo catálogo canônico.

A massa de avaliação permanece inalterada no subconjunto PS atual, sem amplificação sintética adicional. O volume observado é suficiente para os cenários de pergunta definidos neste protocolo.

Por fim, os modelos de linguagem considerados nas corridas são o OpenAI *gpt-5.4-nano* e o Google *gemini-3.5-flash*, fixados como *modelVersion* de referência e registrados por corrida. A decisão por dois provedores busca reduzir o risco de conclusões excessivamente dependentes de um único comportamento de inferência, mantendo rastreabilidade de configuração em cada execução. As tarifas de *tokens* de entrada e saída de cada modelo, utilizadas no cálculo do custo FinOps, são apresentadas na seção de indicadores e agregação.

4.2 Métodos

Os métodos adotados estruturam-se em quatro frentes: modelagem da arquitetura do artefato, sua efetiva implementação, protocolo de avaliação e governança de registro/validade.

O princípio comum entre essas frentes é a auditabilidade de cada corrida experimental, da pergunta inicial ao desfecho classificado conforme as convenções definidas.

4.2.1 Unidade experimental e vocabulário

Para execução do procedimento, adota-se: cada item da bateria gera **uma** pergunta, **uma** sessão e **uma** corrida; a relação pergunta $\rightarrow$ SQL candidato é **1:N**, onde uma pergunta em NL pode originar N consultas SQL na mesma sessão.

- **Pergunta:** enunciado em linguagem natural submetido ao artefato *Text-to-SQL*, com formulação fixada na bateria de avaliação (por exemplo: “*Quais franquias estão ativas no cadastro?*”). Constitui a entrada do experimento, e não o diálogo multi-turno com várias consultas sucessivas do usuário.
- **Sessão:** episódio delimitado de interação do agente orquestrador para processar **uma** pergunta, desde a descoberta de metadados via *tools* MCP até a geração de um ou mais SQL candidatos e o encerramento da trilha registável. O orçamento de *tool calls* aplica-se por sessão, isto é, por pergunta tratada, com limite máximo de 10 chamadas.
- **Corrida:** execução experimental auditável da cadeia pergunta $\rightarrow$ *tools* MCP $\rightarrow$ SQL(s) gerada(s) $\rightarrow$ validação $\rightarrow$ execução $\rightarrow$ registro, identificada por um *runId* único e arquivada em diretório dedicado de evidências. A corrida materializa a sessão no *harness* e é a unidade sobre a qual se calculam métricas e desfechos (*success*, *partial_success*, *syntax_error*, *structural_error*, *execution_error*, *budget_exceeded*).

Não se confunde **sessão** ou **corrida** com o processo servidor MCP em execução contínua no ambiente laboratorial: este permanece ativo entre perguntas, mas cada avaliação experimental reinicia o ciclo com identificador e registro próprios.

Cada pergunta da bateria pode originar até cinco consultas SQL ($n_i \leq 5$) na mesma sessão. A trilha convencionada *session.jsonl* registra a lista completa de *statements* e o desfecho agregado por corrida; a resposta apresentada ao avaliador é **agregada**, abrangendo todos os SQLs gerados na sessão.

4.2.2 Arquitetura do artefato

A arquitetura metodológica do artefato segue uma organização por contextos e responsabilidades, aplicação e integrações. Em termos funcionais, o ciclo experimental é composto por seis passos: recepção da pergunta, descoberta de metadados, síntese de SQL, validação, execução controlada e registro de evidências. O painel (b) da Figura 4 sintetiza essa sequência e explicita a centralidade da descoberta de metadados no protocolo de geração de SQL.

A topologia do artefato distribui-se em dois planos complementares, reunidos na Figura 5. O plano de aplicação, no painel (a), reúne o orquestrador em Spring Boot, organizado segundo arquitetura hexagonal, e o servidor MCP que expõe as *tools* de catálogo. Nesse plano, a recepção da pergunta gera o *runId*, os casos de uso coordenam descoberta, geração,

validação e registro, e os adaptadores isolam o acesso aos provedores de modelo, ao catálogo, ao Apache Calcite e ao executor JDBC em Hive. O plano de dados, no painel (b), reúne o cluster ODP que hospeda o Apache Atlas e a massa PS fixa (92 tabelas) em Hive/HDFS. O Atlas reside no nó *master*, apoiado por HBase, Solr e Kafka distribuídos nos nós *workers*, que também sustentam o armazenamento HDFS com replicação e o motor de consulta Hive sobre a massa inerte. Essa separação evidencia que a descoberta de metadados via MCP e a execução de SQL incidem sobre o mesmo *plano de dados* governado.

Essa cadeia impõe uma restrição importante: o agente não deve produzir SQL sem antes consultar metadados canônicos via MCP. Portanto, a etapa de descoberta não é acessória; ela constitui condição de validade da corrida. Com isso, a geração deixa de ser avaliada apenas por fluência sintática e passa a ser avaliada pela aderência ao catálogo canônico.

Fixam-se três *tools* basilares de catálogo, identificadas como `catalog.listTables` (listagem de tabelas do esquema governado), `catalog.describeTable` (descrição de colunas de uma tabela) e `catalog.listRelationships` (listagem de relacionamentos de uma tabela). Essas três operações cobrem o percurso mínimo de *schema linking*, da seleção de tabelas à recuperação de junções suportadas. O contrato MCP associado define formato de resposta, taxonomia de erros de chamada e política de compatibilidade de versões. Para fins metodológicos, importa menos a estrutura *JavaScript Object Notation* (JSON) literal e mais o fato de que o protocolo torna observável quais metadados foram consultados antes da síntese de SQL.

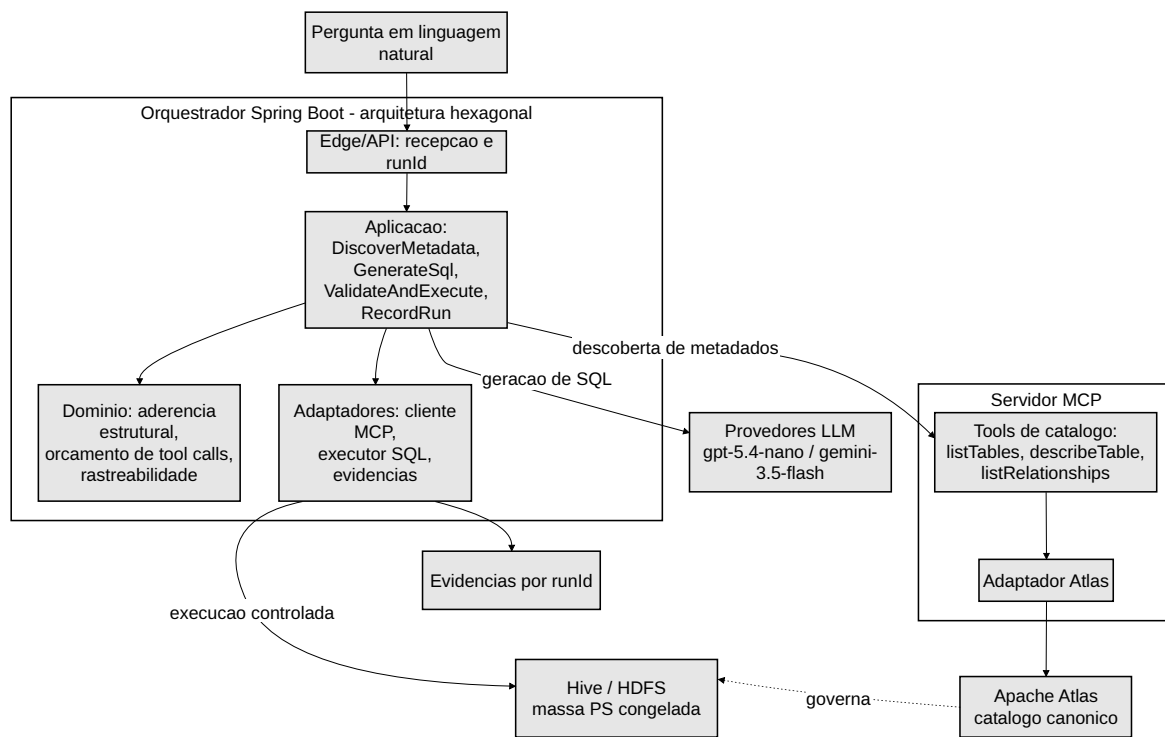
O catálogo MCP possui quinze *tools* de descoberta sobre o Apache Atlas, todas somente leitura e sujeitas ao mesmo contrato de API do componente Apache. As demais doze, com exceção das já citadas três basilares, são complementares e ampliam o *schema linking* sem alterar o contrato nem o orçamento de chamadas. As operações complementares derivam das capacidades da API REST v2 do Apache Atlas, entre elas a busca de entidades, a consulta de linhagem e a leitura de classificações de governança. Todas operam sob o mesmo orçamento de dez chamadas por sessão, de modo que o ganho de *schema linking* de cada chamada passa a ser ponderado pela corrida e não pela disponibilidade da *tool*.

A Tabela 1 apresenta o catálogo completo e a operação correspondente da API REST v2 do Apache Atlas.

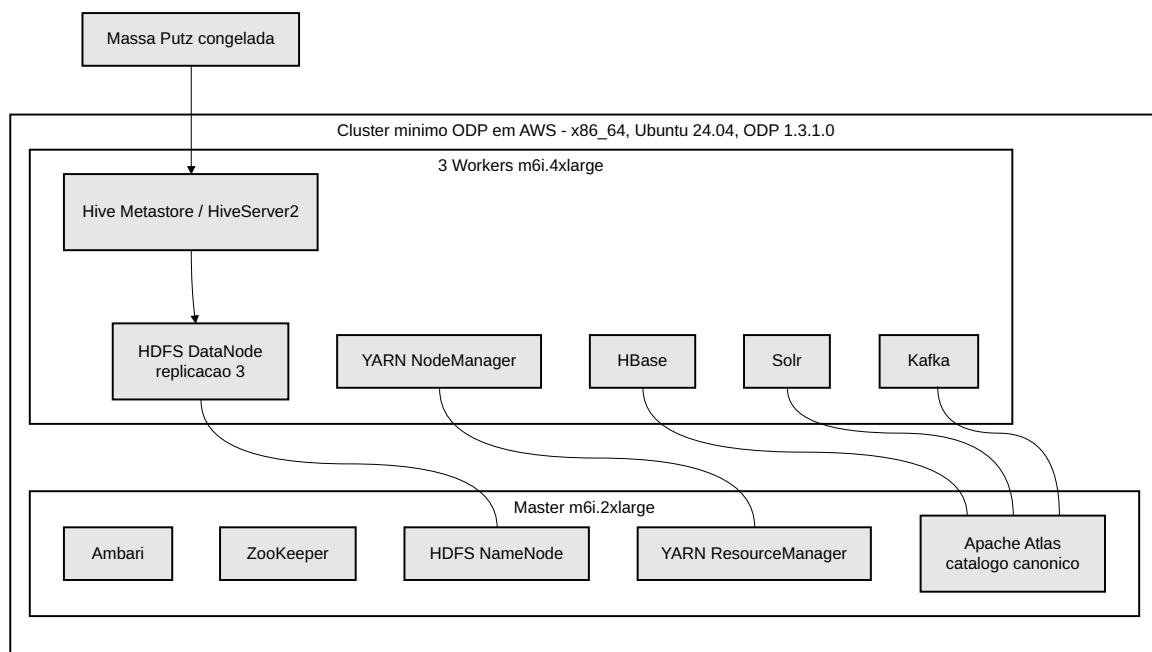
Tabela 1 – Catálogo MCP: quinze *tools* de descoberta sobre o Apache Atlas.

Tool MCP	Função	Operação Atlas (v2)
<code>listTables</code>	Lista tabelas do esquema governado	GET /v2/search/dsl
<code>describeTable</code>	Descreve colunas de uma tabela	GET /v2/entity/uniqueAttr
<code>listRelationships</code>	Lista relacionamentos de uma tabela	GET /v2/types/relationshipdef
<code>searchTables</code>	Busca tabelas por termo da pergunta	GET /v2/search/dsl
<code>quickSearch</code>	Ranqueia candidatos com agregações	GET /v2/search/quick
<code>searchByAttribute</code>	Casa entidades por atributo exato	GET /v2/search/attribute
<code>fullTextSearch</code>	Recupera entidades por texto livre	GET /v2/search/fulltext
<code>getEntityByGuid</code>	Resolve entidade completa por GUID	GET /v2/entity/guid
<code>getEntityByName</code>	Resolve entidade pelo nome canônico	GET /v2/entity/uniqueAttr
<code>getEntityHeader</code>	Confirma existência e tipo da entidade	GET /v2/entity/guid/header
<code>getClassifications</code>	Lê classificações de governança	GET /v2/entity/classifications
<code>getEntityAudit</code>	Lê histórico de auditoria da entidade	GET /v2/entity/audit
<code>getLineage</code>	Recupera linhagem entre tabelas	GET /v2/lineage/guid
<code>getEntityTypeDef</code>	Recupera definição de tipo de entidade	GET /v2/types/entitydef
<code>getRelationshipTypeDef</code>	Recupera definição de relacionamento	GET /v2/types/relationshipdef

Figura 5 – Topologia do artefato: plano de aplicação e plano de dados.



(a) Plano de aplicação: orquestrador Spring Boot e servidor MCP.



(b) Plano de dados: cluster ODP com Apache Atlas, Hive e massa PS.

Fonte: Elaboração própria.

A política de orçamento de *tool calls* é tratada como variável de controle experimental. Neste estudo, fixa-se orçamento de referência em 10 chamadas por sessão (equivalente a 10 chamadas por pergunta), como decisão de processo. O consumo efetivo é registrado em cada corrida para permitir comparações entre execuções.

A definição operacional de **erro estrutural** utilizada neste capítulo mantém-se alinhada ao escopo do trabalho: *considera-se erro estrutural quando a consulta referencia tabela, visão ou coluna ausente do catálogo canônico, quando usa schema incompatível com o catálogo ou quando constrói junções não suportadas pelos relacionamentos recuperados*. Essa definição orienta tanto a validação quanto a classificação de desfecho.

4.2.3 Procedimento de Execução

O procedimento é descrito como sequência metodológica de preparação do experimento, onde os detalhes ainda mais específicos serão demonstrados no Desenvolvimento. As etapas abaixo organizam o percurso mínimo necessário para garantir comparabilidade entre corridas:

1. **Delimitação do subconjunto de dados.** Seleciona-se o subconjunto de 92 tabelas PS, fixa-se a versão do conjunto (*datasetVersion*) e define-se o escopo de cenários de pergunta.
2. **Preparação da base.** A massa é preparada no ambiente de dados e publicada de forma consistente para consulta SQL, preservando integridade referencial e controle de versão.
3. **Registro no catálogo canônico.** As entidades técnicas relevantes são registradas no catálogo canônico para que as *tools* MCP reflitam o estado efetivo dos metadados disponíveis ao agente.
4. **Configuração do ciclo de inferência.** Define-se o provedor de modelo, versão do modelo, orçamento de *tools* e demais parâmetros de corrida em formato reproduzível.
5. **Execução controlada das corridas.** Cada pergunta percorre o fluxo completo de descoberta, síntese, validação e execução, com trilha observável de decisões.
6. **Consolidação de evidências.** Os artefatos de sessão são persistidos por *runId*, permitindo auditoria posterior e agregação de métricas.

Esse encadeamento visa rastreabilidade entre decisão metodológica e resultado observado. O detalhamento técnico de provisionamento de infraestrutura, integração de componentes e automações de carga é tratado no Capítulo de Desenvolvimento.

4.2.4 Protocolo de Avaliação

A unidade de avaliação coincide com a **corrida** definida acima: cada execução completa da cadeia pergunta → *tools* MCP → SQL(s) → validação → execução → registro. O protocolo de execução por corrida é apresentado de forma padronizada:

1. Inicialização da sessão e atribuição de *runId*;
2. Consulta de metadados sob orçamento de chamadas;
3. Geração de até cinco SQLs candidatos por pergunta;
4. Validação sintática de cada *statement* via Apache Calcite (dialetos Hive);
5. Validação estrutural contra catálogo canônico;
6. Execução controlada sobre subconjunto PS (92 tabelas);
7. Classificação do desfecho agregado e registro da trilha completa.

As medidas organizam-se em cinco classes: desfecho e aderência, rastreabilidade da trilha experimental, desempenho operacional, custo FinOps de LLM e estimativa de infraestrutura AWS. A interpretação do ganho da arquitetura mediada por MCP exige comparação com um comparativo simples, cuja definição é apresentada na seção de definição de baseline comparativa, ao final deste capítulo.

4.2.5 Indicadores e Agregação

Para agregação e comparação entre corridas, provedores e versões de modelo, as medidas essenciais admitem representação matemática sobre o conjunto de execuções registradas. Seja $\mathcal{C} = \{c_1, \dots, c_N\}$ o conjunto de corridas de uma campanha experimental, com $|\mathcal{C}| = N$. Para cada corrida c_i , denotam-se o_i o desfecho agregado, k_i o número de *tool calls* consumidas na sessão (contagem usada em $\bar{k}$ e na classe *budget_exceeded*), L_i a latência ponta a ponta em segundos, $n_{in,i}$ e $n_{out,i}$ os tokens de entrada e saída reportados pela API do provedor na corrida, e $m(i)$ a *modelVersion* fixada na corrida, que determina as tarifas aplicadas em C_i .

Seja $S_i = \{s_{i,1}, \dots, s_{i,n_i}\}$ o conjunto de SQLs válidos gerados na sessão ($n_i \leq 5$) e G_i o gabarito associado à pergunta. A classificação de o_i adota seis classes mutuamente exclusivas, definidas pela primeira condição verdadeira na ordem de precedência abaixo:

1. *budget_exceeded*: ocorre quando $k_i > 10$.
2. *syntax_error*: falha de *parse* Calcite em algum SQL antes da validação estrutural.
3. *structural_error*: referência inválida ao catálogo canônico.
4. *execution_error*: falha de Pipeline ou nenhum SQL bate o gabarito G_i na resposta agregada.
5. *partial_success*: parte dos SQLs válidos bate G_i , mas não todos.
6. *success*: pipeline completo e todos os SQLs exigidos batem G_i .

A classe *success* exige validação estrutural, execução sem exceção e aderência ao gabarito. A distribuição de o_i por classe permite calcular taxas de erro complementares, fora do conjunto essencial da tabela.

4.2.5.1 Métricas do Artefato Proposto

Define-se a aderência estrutural como

$$A_{\text{est}} = \frac{1}{N} |\{ c_i \in \mathcal{C} : o_i \neq \text{structural_error} \}|, \quad (4.1)$$

a executabilidade como

$$A_{\text{exec}} = \frac{1}{N} |\{ c_i \in \mathcal{C} : o_i = \text{success} \}|, \quad (4.2)$$

e a taxa de sucesso parcial como

$$A_{\text{part}} = \frac{1}{N} |\{ c_i \in \mathcal{C} : o_i = \text{partial_success} \}|. \quad (4.3)$$

Em termos operacionais, A_{est} mede a proporção de corridas sem erro estrutural, A_{exec} a proporção classificada como `success` e A_{part} a proporção com `partial\_success`.

A rastreabilidade da trilha considera trilha(c_i) completa quando o registro em `session.jsonl` cobre pergunta $\rightarrow$ `tools` $\rightarrow$ metadados $\rightarrow$ todos os SQLs de $S_i \rightarrow$ desfechos por `statement`:

$$T = \frac{1}{N} |\{ c_i \in \mathcal{C} : \text{trilha}(c_i) \text{ completa} \}|. \quad (4.4)$$

O indicador T resume, na escala $[0, 1]$, quantas corridas preservaram trilha auditável completa no registro experimental.

O orçamento médio de `tool calls` e a fração de corridas dentro do limite de dez chamadas por sessão são

$$\bar{k} = \frac{1}{N} \sum_{i=1}^N k_i, \quad (4.5)$$

$$R_{\text{budget}} = \frac{1}{N} |\{ c_i \in \mathcal{C} : k_i \leq 10 \}|. \quad (4.6)$$

O valor k_i registra o consumo de `tool calls` na corrida i . A média $\bar{k}$ resume esse consumo na campanha, enquanto R_{budget} indica a fração de corridas que permaneceu no limite de dez chamadas.

Ordenando $\{L_i\}$ como $L_{(1)} \leq \dots \leq L_{(N)}$, reportam-se os percentis

$$L_{p50} = L_{(\lceil 0,5 N \rceil)}, \quad L_{p95} = L_{(\lceil 0,95 N \rceil)}. \quad (4.7)$$

Para cada corrida, L_i mede o tempo total da sessão em segundos, do início do processamento da pergunta ao registro final. Ordenados os valores $\{L_i\}$, o percentil L_{p50} resume o tempo típico da campanha e L_{p95} captura corridas mais lentas na cauda da distribuição.

O custo FinOps por sessão (por corrida) considera apenas tokens LLM, sem custo monetário de `tool calls` Atlas no MVP. Com tarifas $p_{\text{in},m}$ e $p_{\text{out},m}$ por `model/Version` m . No modelo OpenAI gpt-5.4-nano, aplicam-se \$0,20 e \$1,25 por milhão de tokens de entrada e de saída. No modelo Google gemini-3.5-flash, aplicam-se \$1,50 e \$9,00 por milhão:

$$C_i = n_{\text{in},i} p_{\text{in},m(i)} + n_{\text{out},i} p_{\text{out},m(i)}. \quad (4.8)$$

Os tokens $n_{in,i}$ e $n_{out,i}$ são reportados pela API do provedor em cada corrida e entram diretamente em C_i . A versão $m(i)$ seleciona as tarifas vigentes naquele registro. Além de C_i , a campanha pode reportar $\bar{C}$, custo médio por corrida, e C_{total} , soma dos custos de todas as corridas MCP. Esses agregados permanecem fora da Tabela 2.

4.2.5.2 Métricas da Baseline Comparativa e Infraestrutura

No modo *baseline* (injeção estática de esquema, sem MCP), $k_i = 0$ e não se calculam A_{est} , T nem R_{budget} . Seja N_{base} o número de corridas da campanha *baseline-static*, executada sobre a mesma bateria de perguntas. Define-se $g_i = 1$ quando a resposta agregada coincide com G_i e $g_i = 0$ caso contrário:

$$A_{gab} = \frac{1}{N_{base}} \sum_{i=1}^{N_{base}} g_i. \quad (4.9)$$

O indicador A_{gab} mede a proporção de respostas do comparativo simples que coincidem com o gabarito G_i da pergunta. A estimativa C_{infra} refere-se ao custo do cluster AWS no período da campanha MCP. O valor é consolidado ao final da execução com base no tempo de uso das instâncias provisionadas, sem entrar em C_i . Latência, tokens e FinOps aplicam-se apenas ao modo MCP.

A Tabela 2 resume os indicadores essenciais por modo. O símbolo o_i condensa a taxonomia de seis classes definida acima, com precedência fixa na classificação de cada corrida.

Tabela 2 – Indicadores essenciais por modo experimental.

Símbolo	Nome	Unidade	Modo
o_i	Desfecho agregado	enum (6 classes)	MCP
A_{est}	Aderência estrutural	[0,1]	MCP
A_{exec}	Executabilidade	[0,1]	MCP
A_{part}	Sucesso parcial	[0,1]	MCP
T	Rastreabilidade	[0,1]	MCP
$\bar{k}$	Orçamento médio de <i>tool calls</i>	adimensional	MCP
R_{budget}	Fração de corridas dentro do orçamento	[0,1]	MCP
L_{p50}	Latência (percentil 50)	s	MCP
L_{p95}	Latência (percentil 95)	s	MCP
$n_{in,i}$	Tokens de entrada	tokens	MCP
$n_{out,i}$	Tokens de saída	tokens	MCP
C_i	Custo FinOps por corrida	USD	MCP
A_{gab}	Aderência ao gabarito	[0,1]	baseline
C_{infra}	Custo cluster AWS	USD	infraestrutura

A taxonomia de vereditos distingue desfecho de execução de acerto de intenção. Uma consulta pode atravessar a validação sintática e estrutural e executar sem exceção no Hive, mas retornar dados que não respondem à pergunta formulada. Por isso a classe *success* não se apoia apenas na execução bem-sucedida, e sim na coincidência da resposta agregada com o gabarito G_i da pergunta. Quando nenhum SQL válido bate o gabarito, a corrida é classificada como *execution_error*, e quando apenas parte dos SQLs exigidos bate, como *partial_success*. Essa fixação ao gabarito reduz o risco de um acerto sintático mascarar uma falha lógica.

Para cobertura dos cenários definidos sobre a massa PS estável (92 tabelas), adota-se uma bateria de 30 perguntas, distribuída em três grupos de dificuldade, a saber, **SIMPLES**, **MÉDIA** e **COMPLEXA**, e em **seis cenários** metodológicos (Junção 1:N, Junção em cadeia, Agregação, Filtro temporal, Junção geográfica e Junção cross-domínio). No comparativo simples *baseline-static*, a bateria é submetida ao Google *gemini-3.5-flash* em trinta corridas ($N_{\text{base}} = 30$). No modo MCP, a mesma bateria é submetida aos dois provedores de modelo, o que perfaz até 60 corridas por campanha completa antes de réplicas com *seeds* distintas. O inventário das perguntas é mantido em Apêndice A, junto ao gabarito G_i de cada item.

A Tabela 3 lista os trinta itens da bateria aprovada. O plano de campanhas experimental segue a ordem *baseline-static* $\rightarrow$ modo MCP, com réplicas por *seed* documentadas por campanha.

Tabela 3 – Bateria de perguntas da avaliação.

ID	Cenário	Dificuldade
Q01	Junção 1:N	SIMPLES
Q02	Junção 1:N	SIMPLES
Q03	Junção 1:N	SIMPLES
Q04	Junção 1:N	SIMPLES
Q05	Junção em cadeia	SIMPLES
Q06	Agregação	SIMPLES
Q07	Agregação	SIMPLES
Q08	Filtro temporal	SIMPLES
Q09	Junção geográfica	SIMPLES
Q10	Junção geográfica	SIMPLES
Q11	Junção em cadeia	MÉDIA
Q12	Junção em cadeia	MÉDIA
Q13	Agregação	MÉDIA
Q14	Filtro temporal	MÉDIA
Q15	Junção geográfica	MÉDIA
Q16	Junção cross-domínio	MÉDIA
Q17	Junção cross-domínio	MÉDIA
Q18	Junção cross-domínio	MÉDIA
Q19	Junção cross-domínio	MÉDIA
Q20	Junção cross-domínio	MÉDIA
Q21	Junção em cadeia	COMPLEXA
Q22	Junção em cadeia	COMPLEXA
Q23	Agregação	COMPLEXA
Q24	Agregação	COMPLEXA
Q25	Junção cross-domínio	COMPLEXA
Q26	Junção cross-domínio	COMPLEXA
Q27	Junção cross-domínio	COMPLEXA
Q28	Junção cross-domínio	COMPLEXA
Q29	Filtro temporal	COMPLEXA
Q30	Junção geográfica	COMPLEXA

A escolha por 30 perguntas equilibra cobertura de cenários e custo de execução em duas frentes de provedor, entretanto, impõe limitação estatística. Este conjunto é pequeno frente a benchmarks de larga escala como Spider e BIRD, de modo que os resultados têm caráter de estudo de caso e não sustentam inferência populacional ampla sobre o comportamento dos modelos.

4.2.6 Registro Experimental

A reprodutibilidade depende do registro padronizado de metadados e artefatos por execução. Cada corrida deve registrar, no mínimo, *runId*, *modelVersion*, *seed*, *commitHash*, *datasetVersion* e *contractsVersion*. A composição do pacote por corrida no modo MCP resume-se na Figura 6.

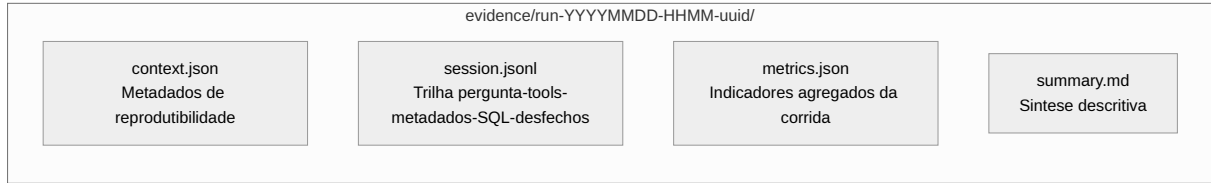


Figura 6 – Pacote de evidências por corrida experimental (modo MCP).

Fonte: Elaboração própria.

A estrutura de evidências adota diretório dedicado por execução. A segregação por *runId* evita sobrescrita de histórico e sustenta auditoria longitudinal da execução experimental.

O protocolo definido nesta seção encerra o desenho experimental como estudo de caso *single-domain* no domínio PS, sem pretensão de generalização para outros domínios. A comparação com Spider e BIRD permanece referencial, e a massa PS serve de prova de coerência estrutural contra catálogo governado, métrica que esses benchmarks não reportam. Materializam-se assim as três contribuições assinaladas no Capítulo de Trabalhos Relacionados. A primeira consiste no registro reprodutível da orquestração decisória, com corrida por *runId*, trilha (pergunta-*tools*-SQL), classificação de desfecho e indicadores versionados por execução. A segunda concretiza o confronto quantitativo entre mediação por MCP e injeção estática do DDL canônico PS no comparativo simples, sob a mesma bateria, *modelVersion* e *seed*. A terceira verifica aderência estrutural contra catálogo governado, métrica distinta da ênfase em execução dos benchmarks empresariais, sem que o trabalho se proponha como novo benchmark de escala.

4.3 Escolhas Arquiteturais

Com os métodos experimentais já explicitados, registram-se as decisões de desenho do artefato e sua posição frente a alternativas da literatura e da prática de engenharia, em continuidade ao Capítulo de Trabalhos Relacionados. Métricas, orçamento de *tool calls* e registro por *runId* permanecem os definidos na Seção de Métodos. Aqui compara-se apenas o que se **adota** ou se **descarta** em cada eixo.

4.3.1 Vinculação Progressiva em Detrimento de Injeção Estática e RAG

O artefato materializa *schema linking* por consultas sucessivas tipadas ao catálogo, e não por injeção opaca de esquema no *prompt*. Abordagens de *few-shot*, DIN-SQL (POURREZA; RAFIEI, 2023) e MCI-SQL (WANG et al., 2025) elevam acurácia quando o esquema cabe no orçamento de *tokens*, porém dificultam registrar quais partes do esquema foram consideradas em cada corrida.

TRUST-SQL (JIAN et al., 2026) compartilha a linha de exploração progressiva. Neste trabalho, a diferença metodológica é o catálogo canônico e os contratos MCP fixados. RAG e *Graph RAG* vetoriais (MOHAMMADJAFARI; MAIDA; GOTTUMUKKALA, 2025) podem complementar glossários de negócio, mas não substituem inventário estrutural de tabelas e junções; permanecem fora do MVP. Adota-se **vinculação progressiva mediada por catálogo**, de modo que a aderência seja verificável contra o inventário canônico. A Tabela 4 resume o eixo.

Tabela 4 – Mecanismos de contexto e decisão.

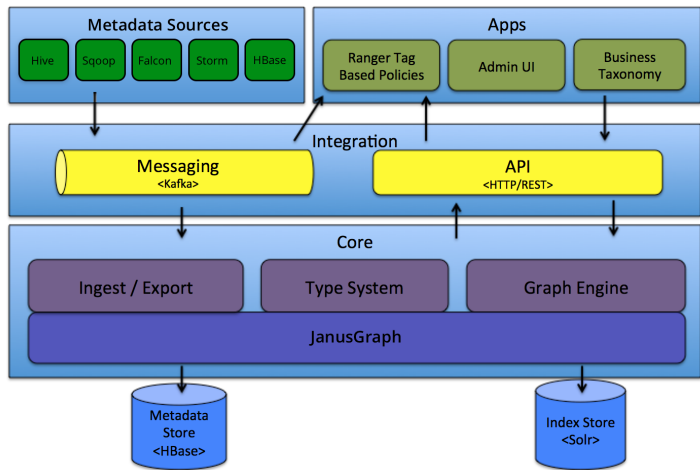
Abordagem	Grounding estrutural	Auditabilidade	Decisão
Esquema no <i>prompt</i> (few-shot, DIN-SQL, MCI-SQL)	Parcial	Baixa	Não adotada
<i>Schema retrieval</i> estático	Parcial	Baixa	Não adotada
Vinculação progressiva e <i>tools MCP</i>	Alta	Alta	Adotada

4.3.2 Apache Atlas frente a Fontes Paralelas de Esquema

A vinculação progressiva exige fonte única de informação sobre o subconjunto de esquema exposto ao agente (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025). Adota-se **Apache Atlas** no laboratório por integração com entidades após carga PS em HDFS/Hive, por linhagem e classificações no mesmo *data plane* do experimento.

A Figura 7 resume a arquitetura de referência do Apache Atlas no cluster Hadoop. Conectores de metadados (Hive e demais fontes do *stack*) publicam eventos com Kafka. A camada de ingestão alimenta o *type system* e o grafo de entidades (JanusGraph), persistido em HBase e indexado em Solr. Aplicações de governança e a API HTTP/REST consomem o mesmo núcleo.

Figura 7 – Arquitetura de referência do Apache Atlas no ecossistema ODP.



Fonte: Adaptado de The Apache Software Foundation (2025).

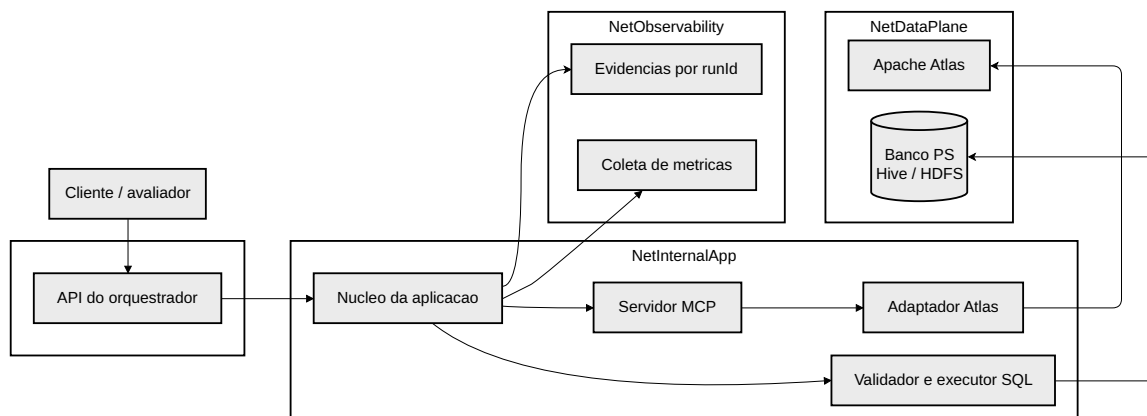
Por decisão de implementação, comparações de mercado como Atlan, DataHub, OpenMetadata, Data-bricks, Amazon EMR entre outros, ficam fora do MVP.

4.3.3 Cluster ODP e Massa Real frente a Recortes Simplificados

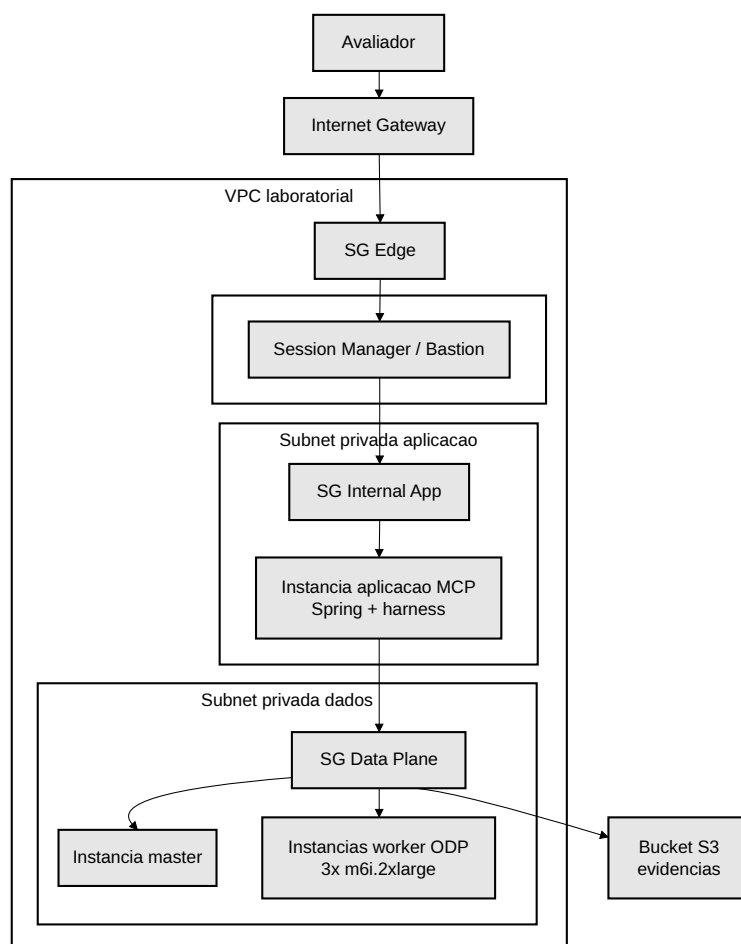
O Atlas co-reside com dependências de outros componentes no cluster (Hive, HBase, Solr, Kafka). Um SGBD isolado na aplicação não reproduziria *data lake* com governança nem a carga PS em HDFS com Atlas integrado. Adota-se cluster mínimo em AWS (x86_64, Ubuntu 24.04, ODP 1.3.1.0): um *master* e três *workers*, Atlas no *master*, massa PS estável (92 tabelas) em Hive/HDFS.

A separação entre o serviço Web Spring e o cluster de dados é demonstrada pelas zonas lógicas de rede e na implantação AWS reunidas na Figura 8. O painel (a) distribui os componentes do artefato em NetPublic, NetInternalApp, NetDataPlane e NetObservability, evidenciando o encadeamento API → MCP → Atlas → Hive antes do registro de evidências. O painel (b) traduz essa organização para a VPC (Virtual Private Cloud) laboratorial: subnets pública e privada, *security groups*, instância dedicada da aplicação MCP separada do *master* e dos *workers* ODP, e bucket S3 para trilhas por *runId*.

Figura 8 – Topologia de rede: zonas lógicas e implantação AWS.



(a) Zonas lógicas: Spring, MCP, Atlas e banco PS.



(b) VPC AWS: aplicação MCP, cluster ODP e evidências S3.

Fonte: Elaboração própria.

A justificativa pelo uso do cluster é a infraestrutura de dados que sustenta esse catálogo: armazenamento distribuído em HDFS, motor de consulta Hive e os serviços de apoio do Atlas. Um catálogo em arquivo ou um SGBD relacional isolado forneceria esquema ao *prompt*, mas não reproduziria a governança e a linhagem do *data plane* real, e o Atlas em modo *standalone* não exercitaria a integração com Hive que caracteriza o cenário corporativo. Reconhece-se que essa abordagem acrescenta latência e complexidade operacional frente a essas possibilidades. Esse custo é assumido porque o objeto de estudo não é apenas a tarefa de PLN em si, mas também o estudo arquitetural associado à essa tarefa.

4.3.4 Protocolo do Comparativo Simples para Baseline

A avaliação do ganho da arquitetura mediada por MCP exige um comparativo simples que isole o efeito da descoberta de metadados. Por isso, define-se uma campanha *baseline* comparativa, executada antes do modo MCP, cujos resultados servem de referência para a discussão de Desenvolvimento.

O comparativo simples reproduz o ciclo pergunta $\rightarrow$ SQL $\rightarrow$ validação $\rightarrow$ execução sem qualquer descoberta via *tools* MCP. O esquema relacional PS (92 tabelas) é fornecido por injeção estática, anexado integralmente ao *prompt* do modelo. Não há chamada de ferramenta, de modo que o orçamento de *tool calls* é nulo e a avaliação ocorre no ambiente do provedor Google, sem infraestrutura local de catálogo.

A campanha inicial do comparativo simples usa apenas *gemini-3.5-flash*, com $N_{\text{base}} = 30$ corridas (uma por pergunta). Compartilha com o modo MCP a mesma bateria de perguntas e a mesma *datasetVersion* da massa fixa. A variação controlada é a ausência da descoberta mediada por MCP. A campanha é identificada por *campaignId* *baseline-static*, em contraste com o *campaignId* *mcp* do modo de descoberta.

Conforme a distinção de métricas por modo, a taxonomia de seis classes de desfecho, conforme Figura 6, não se aplica ao comparativo simples, pois não há trilha de *tools* nem aderência estrutural verificável contra catálogo governado. O indicador essencial do comparativo simples é a aderência ao gabarito A_{gab} .

4.3.5 Execução e Registro da Campanha

A campanha do comparativo simples é executada manualmente no ambiente Google e precede o modo MCP, em coerência com a ordem adotada na metodologia.

Antes da campanha, as colas G_i do gabarito (Apêndice A e) foram auditadas quanto à executabilidade no MySQL sobre a massa de teste. A Tabela 5 consolida o registro da campanha.

Tabela 5 – Registro da campanha *baseline-static*.

Indicador	Valor registrado
Modelo (<i>campaignId</i>)	Google <i>gemini-3.5-flash</i>
Corridas N_{base}	30
Colas G_i com defeito SQL inicial	11 de 30 (36,7%)
A_{gab} (resposta vs. G_i)	19 de 30 (63,3%)

A agregação de A_{gab} deriva do CSV de evidência, segundo a equação 4.9. A comparação pareada entre os modos *baseline-static* e *mcp* é retomada no Capítulo de Desenvolvimento, onde a campanha MCP é conduzida.

Em síntese, o capítulo metodológico estabelece um desenho experimental auditável, explicita métodos, condensa as escolhas arquiteturais em quatro eixos de *trade-off* (contexto estrutural, catálogo, integração MCP e ambiente de avaliação) e define o paralelo simples que dá sentido comparativo às métricas. A materialização técnica da arquitetura, da infraestrutura e das integrações, bem como a campanha MCP pareada ao *baseline*, é apresentada no Capítulo de Desenvolvimento.

5 Cronograma de execução

Este capítulo consolida o **plano de execução** do trabalho no período de implementação experimental e redação, de **julho a novembro de 2026**, alinhado ao procedimento descrito no Capítulo 4.

Abaixo, cada atividade está relacionada aos meses em que está prevista.

Tabela 6 – Cronograma proposto.

Atividade	Jul.	Ago.	Set.	Out.	Nov.
Massa de teste, ambiente e reprodutibilidade	×	–	–	–	–
Camada semântica / catálogo canônico (Atlas) e adaptador	×	×	–	–	–
Servidor MCP e ferramentas de consulta	–	×	×	–	–
Integração do agente, geração de SQL e validação	–	–	×	×	–
Corridas experimentais e consolidação de métricas	–	–	–	×	×
Redação e revisão dos capítulos da monografia	×	×	×	×	×
Figuras, referências bibliográficas e formatação final	–	–	×	×	×
Apresentação (slides) e preparação da defesa	–	–	–	–	×

Apêndices

APÊNDICE A – Bateria de avaliação e gabarito SQL

Este apêndice consolida a massa de perguntas e o gabarito SQL de referência da bateria *batteryVersion v1*. As fontes canônicas estão em `artifact-docs/evidence/`. O dialeto das colas é MySQL, conforme o DDL canônico `putz_db.sql`.

A.1 Massa de perguntas

A Tabela 7 agrupa os trinta enunciados por dificuldade e cenário metodológico. Cada item gera uma corrida experimental no protocolo descrito no Capítulo de Metodologia.

Tabela 7 – Massa de perguntas da bateria (*batteryVersion v1*).

ID	Cenário	Enunciado
<i>Grupo SIMPLES</i>		
Q01	Junção 1:N	“Para cada franquia, liste suas unidades com a cidade e o estado.”
Q02	Junção 1:N	“Liste cada produto ativo com o nome do seu tipo de produto.”
Q03	Junção 1:N	“Liste os pagamentos de cada projeto, com o projeto, o valor e a data.”
Q04	Junção 1:N	“Quantas franquias estão associadas a cada rede de franquia?”
Q05	Junção em cadeia	“Relacione cada franquia ao seu segmento e à sua rede correspondente.”
Q06	Agregação	“Qual o valor total de pagamentos por projeto?”
Q07	Agregação	“Quantos projetos existem em cada status?”
Q08	Filtro temporal	“Qual a soma do valor das transações com sucesso por mês em 2026?”
Q09	Junção geográfica	“Para cada unidade federativa, quais cidades estão cadastradas?”
Q10	Junção geográfica	“Quantas cidades estão cadastradas por unidade federativa?”
<i>Grupo MÉDIA</i>		
Q11	Junção em cadeia	“Para cada item de projeto, mostre o projeto, o produto e o freelancer responsável.”
Q12	Junção em cadeia	“Liste os eventos de timeline aprovados, com o projeto e a etapa correspondentes.”
Q13	Agregação	“Quais projetos têm total de pagamentos acima da média de pagamentos por projeto?”
Q14	Filtro temporal	“Quantas transações com sucesso ocorreram no primeiro e no segundo semestre de 2026?”
Q15	Junção geográfica	“Qual o valor total de transações com sucesso por unidade federativa do cliente?”
Q16	Junção cross-domínio	“Para transações de sucesso ligadas a itens de projeto, mostre valor, projeto, produto e cliente.”
Q17	Junção cross-domínio	“Para cada cupom usado em um projeto, mostre projeto, tipo e status do cupom, desconto e quem usou.”

Tabela 7 – Massa de perguntas da bateria (*batteryVersion* v1) (continuação).

ID	Cenário	Enunciado
Q18	Junção cross-domínio	<i>“Quais clientes realizaram transações com sucesso e qual o total movimentado por cliente?”</i>
Q19	Junção cross-domínio	<i>“Para cada tipo de produto, quantos itens de projeto existem e qual a soma do valor base?”</i>
Q20	Junção cross-domínio	<i>“Liste as assinaturas ativas com o plano, a categoria do plano e o valor da transação de pagamento.”</i>
<i>Grupo COMPLEXA</i>		
Q21	Junção em cadeia	<i>“Para cada item de ativo ativo, mostre o nome do item, a sua categoria e a categoria-raiz correspondente.”</i>
Q22	Junção em cadeia	<i>“Liste os itens de ativo do tipo IMAGE com categoria, categoria-raiz e tipo, apenas em categorias públicas.”</i>
Q23	Agregação	<i>“Quantos itens de ativo ativos existem em cada categoria-raiz pública, e qual a raiz com mais itens?”</i>
Q24	Agregação	<i>“Para cada autoridade de permissão, quantos usuários a possuem, com a descrição da autoridade?”</i>
Q25	Junção cross-domínio	<i>“Para cada categoria de ativo, quantos usuários têm acesso a ela, com a sua categoria-raiz?”</i>
Q26	Junção cross-domínio	<i>“Para cada autoridade, quantos usuários distintos têm acesso a categorias de ativo públicas?”</i>
Q27	Junção cross-domínio	<i>“Quantos itens de render há no painel para solicitações sem conta e com conta vinculada?”</i>
Q28	Junção cross-domínio	<i>“Para solicitações com item de render e publicação, liste o status, o render e se há conta vinculada.”</i>
Q29	Filtro temporal	<i>“Quantas solicitações de publicação foram criadas por mês em 2026, separando com e sem conta?”</i>
Q30	Junção geográfica	<i>“Quantos itens de render foram solicitados por clientes de cada unidade federativa, com a região?”</i>

A.2 Gabarito SQL de referência

Os blocos a seguir reproduzem a cola SQL de referência (G_i) por pergunta, pronta para execução contra a massa fixa `putz_teste`.

Q01 Listing A.1 – SQL de referência (MySQL) — Q01, cenário Junção 1:N.

```
SELECT f.nome AS franquias, u.cidade, u.estado
FROM franquias f
JOIN franquias_unidades u ON u.franquia_id = f.id
ORDER BY f.nome, u.estado;
```

Q02 Listing A.2 – SQL de referência (MySQL) — Q02, cenário Junção 1:N.

```
SELECT p.name AS produto, pt.name AS tipo
FROM product p
JOIN product_type pt ON p.product_type_id = pt.id
WHERE p.is_active = 1
ORDER BY pt.name, p.name;
```

Q03 Listing A.3 – SQL de referência (MySQL) — Q03, cenário Junção 1:N.

```
SELECT pr.name AS projeto, pp.payment_value, pp.payment_date
FROM project pr
JOIN project_payment pp ON pp.project_id = pr.id
WHERE pp.is_active = 1
ORDER BY pp.payment_date DESC;
```

Q04 Listing A.4 – SQL de referência (MySQL) — Q04, cenário Junção 1:N.

```
SELECT r.nome AS rede, COUNT(f.id) AS total_franquias
FROM franquias__redes_franquia r
LEFT JOIN franquias f ON f.rede_id = r.id
GROUP BY r.id, r.nome
ORDER BY total_franquias DESC;
```

Q05 Listing A.5 – SQL de referência (MySQL) — Q05, cenário Junção em cadeia.

```
SELECT f.nome AS franquias, s.nome AS segmento, r.nome AS rede
FROM franquias f
LEFT JOIN franquias__segmentos s ON f.segmento_id = s.id
LEFT JOIN franquias__redes_franquia r ON f.rede_id = r.id
ORDER BY f.nome;
```

Q06 Listing A.6 – SQL de referência (MySQL) — Q06, cenário Agregação.

```
SELECT pr.name AS projeto, SUM(pp.payment_value) AS total_pago
FROM project pr
JOIN project_payment pp ON pp.project_id = pr.id
WHERE pp.is_active = 1
GROUP BY pr.id, pr.name
ORDER BY total_pago DESC;
```

Q07 Listing A.7 – SQL de referência (MySQL) — Q07, cenário Agregação.

```
SELECT project_status, COUNT(*) AS total
FROM project
GROUP BY project_status
ORDER BY total DESC;
```

Q08 Listing A.8 – SQL de referência (MySQL) — Q08, cenário Filtro temporal.

```
SELECT MONTH(created_date) AS mes, SUM(amount) AS total
FROM transaction
WHERE status = 'SUCCESS'
      AND YEAR(created_date) = 2026
GROUP BY MONTH(created_date)
ORDER BY mes;
```

Q09 Listing A.9 – SQL de referência (MySQL) — Q09, cenário Junção geográfica.

```
SELECT uf.uf, c.name AS cidade
```

```
FROM ibge__uf uf
JOIN ibge__city c ON c.uf_id = uf.id
WHERE c.is_active = 1
ORDER BY uf.uf, c.name;
```

Q10 Listing A.10 – SQL de referência (MySQL) — Q10, cenário Junção geográfica.

```
SELECT uf.uf, COUNT(c.id) AS total_cidades
FROM ibge__uf uf
JOIN ibge__city c ON c.uf_id = uf.id
GROUP BY uf.id, uf.uf
ORDER BY total_cidades DESC;
```

Q11 Listing A.11 – SQL de referência (MySQL) — Q11, cenário Junção em cadeia.

```
SELECT p.name AS projeto, prod.name AS produto, fr.name AS freelancer
FROM project_item pi
JOIN project p ON pi.project_id = p.id
JOIN product prod ON pi.product_id = prod.id
JOIN person fr ON pi.freelancer_id = fr.id
WHERE pi.is_active = 1
ORDER BY p.name;
```

Q12 Listing A.12 – SQL de referência (MySQL) — Q12, cenário Junção em cadeia.

```
SELECT p.name AS projeto, ps.name AS etapa, te.title AS evento, te.conclusion_date
FROM timeline_event te
JOIN project_step ps ON te.project_step_id = ps.id
JOIN project p ON ps.project_id = p.id
WHERE te.event_type = 'APPROVED'
ORDER BY te.conclusion_date DESC;
```

Q13 Listing A.13 – SQL de referência (MySQL) — Q13, cenário Agregação.

```
SELECT pr.name AS projeto, SUM(pp.payment_value) AS total_pago
FROM project pr
JOIN project_payment pp ON pp.project_id = pr.id
WHERE pp.is_active = 1
GROUP BY pr.id, pr.name
HAVING SUM(pp.payment_value) > (
    SELECT AVG(soma)
    FROM (
        SELECT SUM(payment_value) AS soma
        FROM project_payment
        WHERE is_active = 1
        GROUP BY project_id
    ) AS medias
)
ORDER BY total_pago DESC;
```

Q14 Listing A.14 – SQL de referência (MySQL) — Q14, cenário Filtro temporal.

```
SELECT
  CASE
    WHEN MONTH(created_date) <= 6 THEN 'S1'
    ELSE 'S2'
  END AS semestre,
  COUNT(*) AS total
FROM transaction
WHERE status = 'SUCCESS'
  AND YEAR(created_date) = 2026
GROUP BY semestre
ORDER BY semestre
LIMIT 0, 25;
```

Q15 Listing A.15 – SQL de referência (MySQL) — Q15, cenário Junção geográfica.

```
SELECT uf.uf, uf.region, SUM(t.amount) AS total
FROM transaction t
JOIN person pe ON t.person_id = pe.id
JOIN ibge_uf uf ON pe.address_uf = uf.uf
WHERE t.status = 'SUCCESS'
GROUP BY uf.id, uf.uf, uf.region
ORDER BY total DESC;
```

Q16 Listing A.16 – SQL de referência (MySQL) — Q16, cenário Junção cross-domínio.

```
SELECT t.amount, p.name AS projeto, prod.name AS produto, cli.name AS cliente
FROM transaction t
JOIN project_item pi ON t.project_item_id = pi.id
JOIN project p ON pi.project_id = p.id
JOIN product prod ON pi.product_id = prod.id
JOIN person cli ON p.client_id = cli.id
WHERE t.status = 'SUCCESS'
ORDER BY t.amount DESC;
```

Q17 Listing A.17 – SQL de referência (MySQL) — Q17, cenário Junção cross-domínio.

```
SELECT p.name AS projeto, tc.coupon_type, tc.code_status,
  tc.discount_percentual, usr.name AS usuario_cupom
FROM transaction_coupon tc
JOIN project p ON tc.project_id = p.id
JOIN person usr ON tc.person_id = usr.id
WHERE tc.code_status = 'USED'
ORDER BY tc.discount_percentual DESC;
```

Q18 Listing A.18 – SQL de referência (MySQL) — Q18, cenário Junção cross-domínio.

```
SELECT pe.name AS cliente, COUNT(DISTINCT t.project_id) AS projetos,
  SUM(t.amount) AS total
FROM transaction t
JOIN person pe ON t.person_id = pe.id
```

```
WHERE t.status = 'SUCCESS'
GROUP BY pe.id, pe.name
ORDER BY total DESC;
```

Q19 Listing A.19 – SQL de referência (MySQL) — Q19, cenário Junção cross-domínio.

```
SELECT pt.name AS tipo_produto, COUNT(pi.id) AS total_itens,
       SUM(pi.value_base) AS soma_valor_base
FROM project_item pi
JOIN product prod ON pi.product_id = prod.id
JOIN product_type pt ON prod.product_type_id = pt.id
WHERE pi.is_active = 1
GROUP BY pt.id, pt.name
ORDER BY soma_valor_base DESC;
```

Q20 Listing A.20 – SQL de referência (MySQL) — Q20, cenário Junção cross-domínio.

```
SELECT s.id, sp.plan_name, sp.plan_category, t.amount AS valor_transacao,
       s.subscription_status
FROM subscriptions s
JOIN subscription_plans sp ON s.subscription_plan_id = sp.id
LEFT JOIN transaction t ON s.transaction_id = t.id
WHERE s.subscription_status = "ACTIVE"
ORDER BY t.amount DESC;
```

Q21 Listing A.21 – SQL de referência (MySQL) — Q21, cenário Junção em cadeia.

```
SELECT ai.name AS item, c.name AS categoria, raiz.name AS categoria_raiz
FROM asset_item ai
JOIN asset_item_categories aic ON aic.asset_item_id = ai.id
JOIN asset_category c ON aic.category_id = c.id
LEFT JOIN asset_category raiz ON c.root_category_id = raiz.id
WHERE ai.is_active = 1
ORDER BY raiz.name, c.name, ai.name;
```

Q22 Listing A.22 – SQL de referência (MySQL) — Q22, cenário Junção em cadeia.

```
SELECT ai.name AS item, c.name AS categoria, c.category_type AS tipo_categoria,
       raiz.name AS categoria_raiz
FROM asset_item ai
JOIN asset_item_categories aic ON aic.asset_item_id = ai.id
JOIN asset_category c ON aic.category_id = c.id
LEFT JOIN asset_category raiz ON c.root_category_id = raiz.id
WHERE ai.resource_type = 'IMAGE'
      AND c.is_public = 1
ORDER BY raiz.name, c.name, ai.name;
```

Q23 Listing A.23 – SQL de referência (MySQL) — Q23, cenário Agregação.

```
SELECT raiz.name AS categoria_raiz, COUNT(ai.id) AS total_itens
FROM asset_category raiz
```

```

JOIN asset_category c ON c.root_category_id = raiz.id
JOIN asset_item_categories aic ON aic.category_id = c.id
JOIN asset_item ai ON aic.asset_item_id = ai.id
WHERE raiz.is_public = 1 AND ai.is_active = 1
GROUP BY raiz.id, raiz.name
ORDER BY total_itens DESC;

```

Q24

Listing A.24 – SQL de referência (MySQL) — Q24, cenário Agregação.

```

SELECT a.name AS autoridade, a.description, COUNT(ua.user_id) AS total_usuarios
FROM putz_authority a
LEFT JOIN putz_user_authority ua ON ua.authority_name = a.name
GROUP BY a.name, a.description
ORDER BY total_usuarios DESC;

```

Q25

Listing A.25 – SQL de referência (MySQL) — Q25, cenário Junção cross-domínio.

```

SELECT c.name AS categoria, raiz.name AS categoria_raiz,
       COUNT(acu.person_id) AS total_usuarios
FROM asset_category c
LEFT JOIN asset_category raiz ON c.root_category_id = raiz.id
LEFT JOIN asset_category__users acu ON acu.categories_id = c.id
GROUP BY c.id, c.name, raiz.name
ORDER BY total_usuarios DESC;

```

Q26

Listing A.26 – SQL de referência (MySQL) — Q26, cenário Junção cross-domínio.

```

SELECT ua.authority_name AS autoridade, COUNT(DISTINCT u.id) AS usuarios_com_acesso
FROM putz_user_authority ua
JOIN putz_user u ON ua.user_id = u.id
JOIN asset_category__users acu ON acu.person_id = u.id
JOIN asset_category c ON acu.categories_id = c.id
WHERE c.is_public = 1
GROUP BY ua.authority_name
ORDER BY usuarios_com_acesso DESC;

```

Q27

Listing A.27 – SQL de referência (MySQL) — Q27, cenário Junção cross-domínio.

```

SELECT
  CASE
    WHEN pu.owner_id IS NULL THEN 'SEM_CONTA'
    ELSE 'COM_CONTA'
  END AS situacao,
  COUNT(DISTINCT pri.id) AS total_itens_render
FROM publications_user pu
JOIN project_render_item pri ON pu.project_render_item_id = pri.id
JOIN project_render pr ON pri.render_project_id = pr.id
GROUP BY situacao
ORDER BY situacao;

```

Q28

Listing A.28 – SQL de referência (MySQL) — Q28, cenário Junção cross-domínio.

```
SELECT
    pu.status,
    pr.name AS render,
    pub.platform,
    CASE
        WHEN pu.owner_id IS NULL THEN 'SEM_CONTA'
        ELSE 'COM_CONTA'
    END AS situacao
FROM publications_user pu
JOIN project_render_item pri ON pu.project_render_item_id = pri.id
JOIN project_render pr ON pri.render_project_id = pr.id
JOIN publications pub ON pu.publication_id = pub.id
ORDER BY pu.status;
```

Q29

Listing A.29 – SQL de referência (MySQL) — Q29, cenário Filtro temporal.

```
SELECT MONTH(pu.created_date) AS mes,
    SUM(CASE WHEN pu.owner_id IS NULL THEN 1 ELSE 0 END) AS sem_conta,
    SUM(CASE WHEN pu.owner_id IS NOT NULL THEN 1 ELSE 0 END) AS com_conta
FROM publications_user pu
JOIN project_render_item pri ON pu.project_render_item_id = pri.id
JOIN project_render pr ON pri.render_project_id = pr.id
WHERE YEAR(pu.created_date) = 2026
GROUP BY MONTH(pu.created_date)
ORDER BY mes;
```

Q30

Listing A.30 – SQL de referência (MySQL) — Q30, cenário Junção geográfica.

```
SELECT uf.uf, uf.region, COUNT(pri.id) AS total_itens_render
FROM project_render_item pri
JOIN person pe ON pri.person_id = pe.id
JOIN ibge_uf uf ON pe.address_uf = uf.uf
JOIN project_render pr ON pri.render_project_id = pr.id
GROUP BY uf.id, uf.uf, uf.region
ORDER BY total_itens_render DESC;
```

Referências

- ANDROUTSOPOULOS, I.; RITCHIE, G. D.; THANISCH, P. *Natural Language Interfaces to Databases - An Introduction*. 1995. Disponível em: <<https://arxiv.org/abs/cmp-lg/9503016>>. Citado na página 11.
- ANTHROPIC; Model Context Protocol contributors. *Model Context Protocol*. 2024. Especificação aberta para integração de modelos de linguagem com ferramentas e contexto externo. Acesso em maio de 2026. Disponível em: <<https://modelcontextprotocol.io/>>. Citado nas páginas 14 e 15.
- BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014. Citado nas páginas 12 e 14.
- CHATURVEDI, S.; CHADHA, A.; BINDSCHAEDLER, L. *SQL-of-Thought: Multi-agentic Text-to-SQL with Guided Error Correction*. 2025. Disponível em: <<https://arxiv.org/abs/2509.00581>>. Citado na página 21.
- DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd. ed. Basking Ridge, New Jersey, USA: Technics Publications, 2017. Senior Editor: Deborah Henderson. Dedicated to the memory of Patricia Cupoli. ISBN 9781634622349. Disponível em: <<https://www.TechnicsPub.com>>. Citado na página 17.
- FATHOLLAHZADEH, S.; MANSOUR, E.; BOEHM, M. Catdb: Data-catalog-guided, llm-based generation of data-centric ml pipelines. *Proceedings of the VLDB Endowment (PVLDB)*, v. 18, n. 8, p. 2639–2652, 2025. Disponível em: <<https://doi.org/10.14778/3742728.3742754>>. Citado nas páginas 17, 20, 21, 23 e 34.
- GUO, J. et al. *Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation*. 2019. Disponível em: <<https://arxiv.org/abs/1905.08205>>. Citado na página 15.
- HONG, Z. et al. Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL. *IEEE Transactions on Knowledge & Data Engineering*, IEEE Computer Society, Los Alamitos, CA, USA, v. 37, n. 12, p. 7328–7345, dez. 2025. ISSN 1558-2191. Disponível em: <<https://doi.ieeecomputersociety.org/10.1109/TKDE.2025.3609486>>. Citado na página 21.
- JIAN, A. et al. Trust-sql: Tool-integrated multi-turn reinforcement learning for text-to-sql over unknown schemas. *arXiv preprint*, 2026. ArXiv:2603.16448v1. Citado nas páginas 13, 20, 21 e 33.
- LEI, F. et al. *Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows*. 2025. Disponível em: <<https://arxiv.org/abs/2411.07763>>. Citado na página 21.
- LI, J. et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In: OH, A. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 42330–42357. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/file/83fc8fab1710363050bbd1d4b8cc0021-Paper-Datasets_and_Benchmarks.pdf>. Citado nas páginas 12, 20, 21 e 23.
- LIU, M. et al. A systematic review of natural language interfaces for databases. *Frontiers of Computer Science*, Springer, v. 20, n. 11, p. 2011623, 2026. Citado nas páginas 9, 11, 12 e 14.
- MOHAMMADJAFARI, A.; MAIDA, A. S.; GOTTUMUKKALA, R. *From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems*. 2025. Disponível em: <<https://arxiv.org/abs/2410.01066>>. Citado nas páginas 15, 20, 21 e 33.
- POURREZA, M.; RAFIEI, D. *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction*. 2023. Disponível em: <<https://arxiv.org/abs/2304.11015>>. Citado nas páginas 12, 20 e 33.
- RASCHKA, S. *Build a Large Language Model (from Scratch)*. Shelter Island, NY: Manning Publications, 2024. ISBN 978-1633437166. Citado nas páginas 13 e 14.
- SUTSKEVER, I.; VINYALS, O.; LE, Q. V. Sequence to sequence learning with neural networks. *CoRR*, abs/1409.3215, 2014. Disponível em: <<http://arxiv.org/abs/1409.3215>>. Citado na página 11.
- TALAEI, S. et al. *CHESS: Contextual Harnessing for Efficient SQL Synthesis*. 2024. Disponível em: <<https://arxiv.org/abs/2405.16755>>. Citado nas páginas 16, 20 e 21.
- The Apache Software Foundation. *Apache Atlas*. 2025. Documentação do catálogo de metadados e governança. Acesso em maio de 2026. Disponível em: <<https://atlas.apache.org/>>. Citado na página 34.
- VASWANI, A. et al. Attention is all you need. *CoRR*, abs/1706.03762, 2017. Disponível em: <<http://arxiv.org/abs/1706.03762>>. Citado na página 14.

- WANG, B. et al. *MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL*. 2025. Disponível em: <<https://arxiv.org/abs/2312.11242>>. Citado na página 21.
- WANG, B. et al. RAT-SQL: relation-aware schema encoding and linking for text-to-sql parsers. *CoRR*, abs/1911.04942, 2019. Disponível em: <<http://arxiv.org/abs/1911.04942>>. Citado na página 15.
- WANG, Q. et al. Mci-sql: Text-to-sql with metadata-complete context and intermediate correction. *Proceedings of the VLDB Endowment (PVLDB)*, v. 14, n. 1, 2025. Citado nas páginas 13, 17, 20, 21 e 33.
- XIE, T. et al. *UnifiedSKG: Unifying and Multi-Tasking Structured Knowledge Grounding with Text-to-Text Language Models*. 2022. Disponível em: <<https://arxiv.org/abs/2201.05966>>. Citado na página 17.
- YANG, B. et al. Hallucination detection for llm-based text-to-sql generation via two-stage metamorphic testing. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, v. 37, n. 6, 2025. ArXiv:2512.22250. Citado nas páginas 18 e 21.
- YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: RILOFF, E. et al. (Ed.). *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <<https://aclanthology.org/D18-1425/>>. Citado nas páginas 12 e 21.
- ZHA, D. et al. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, ACM New York, NY, v. 57, n. 5, p. 1–42, 2025. Citado na página 16.